ВВЕДЕНИЕ. ОСНОВЫ АНАЛИЗА И ПРОЕКТИРОВАНИЯ ПРОЦЕССОРОВ

Содержание лекции

Часть I. Введение

1. Введение. Структура курса и разные платформы

Часть II. Архитектура и Микроархитектура

1. Архитектура vs Микроархитектура
2. Исторический обзор архитектур. Способы классификации

Часть III. Тренды проектирования

1. Тренды при проектировании вычислительных систем

Часть IV. Система команд (ISA)

1. ISA. Типы данных
2. ISA. Способы адресации операндов

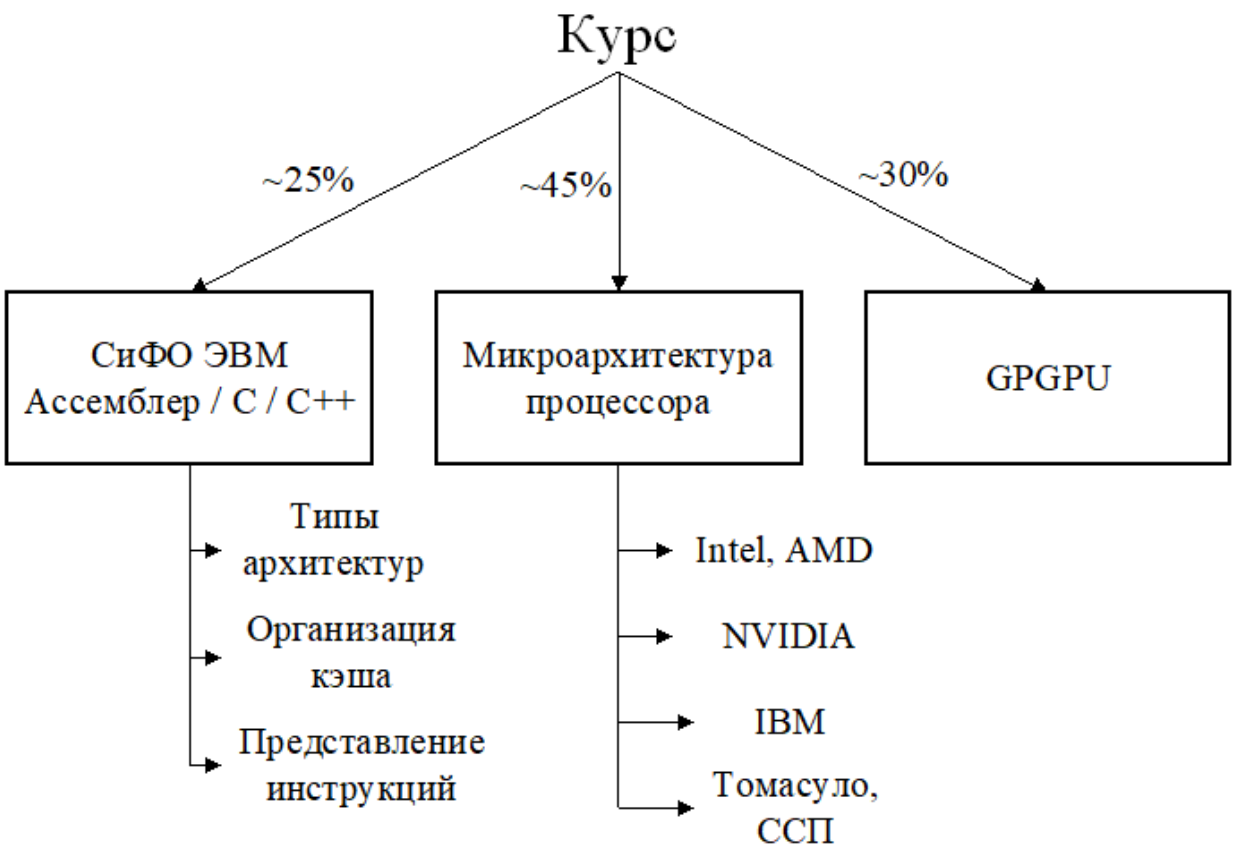
3. ISA. Инструкции
4. Кодирование инструкций

ВВЕДЕНИЕ

Введение

1990'e
Мы скомпилировали и запаковали эту 3D игру с озвучкой и текстурами в файл, размером 1.44Мб, чтобы она смогла запускаться на вашем ПК с 20 МГц камнем и 128 Кб оперативы.
2021
Эта страница с реактивным фреймворком, тремя гифками и парой графиков требует i5 и 8Гб оперативы для плавного скролла.

Структура курса



Что вас ждет

- Лекции: много
- Лаб.работы
  - 6 обязательных
  - начисляются бонусные баллы
- Экзамен
  - максимальный балл: не более +4 к заходной оценке
- Промежуточные тесты
  - по желанию
  - на лаб.работах
  - ТОЛЬКО очно

Разные платформы, разные цели

Разные платформы, разные цели

[5 awesome uses for drone technology]

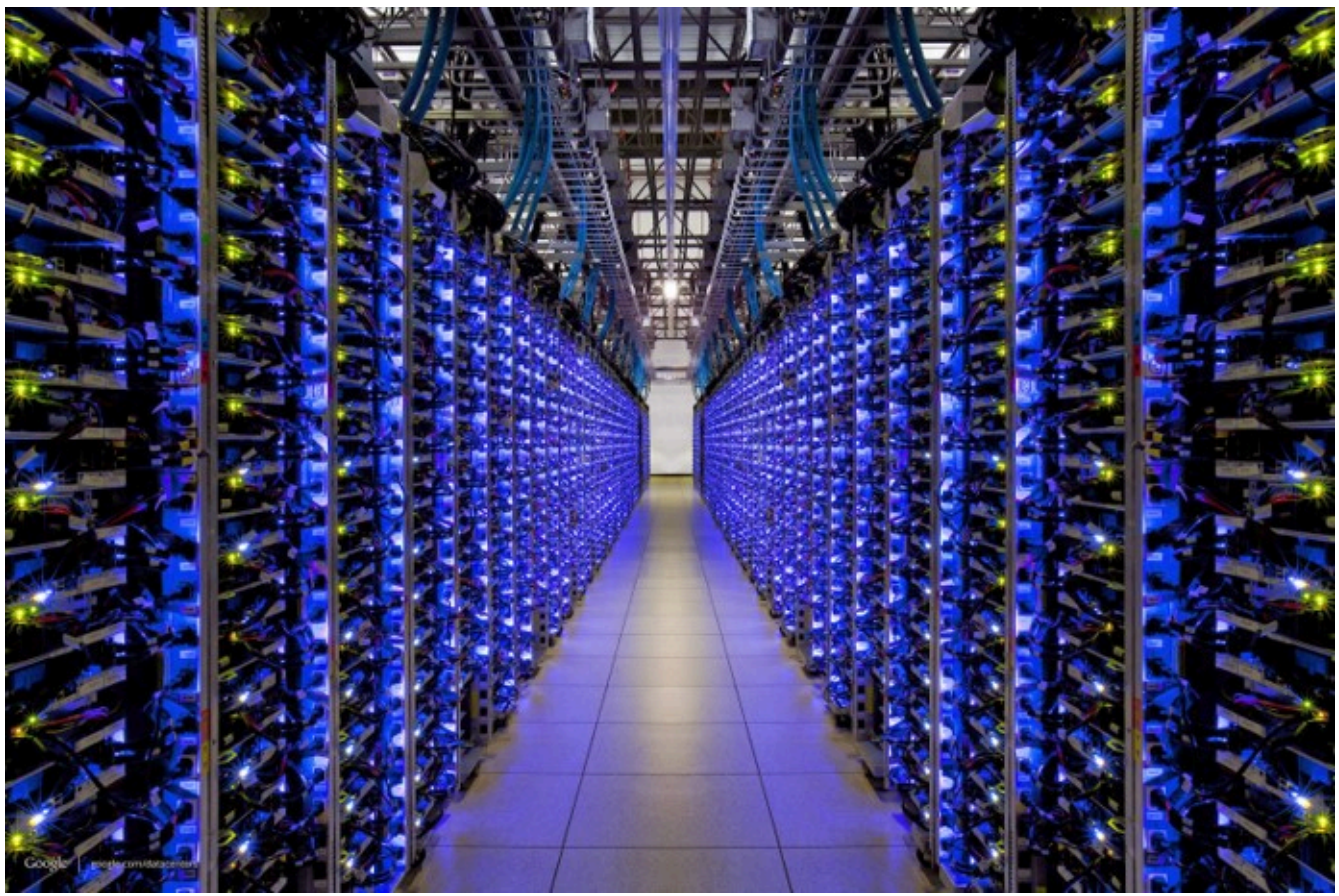
Разные платформы, разные цели

Источник данных

Разные платформы, разные цели

Источник данных

Разные платформы, разные цели



Источник данных

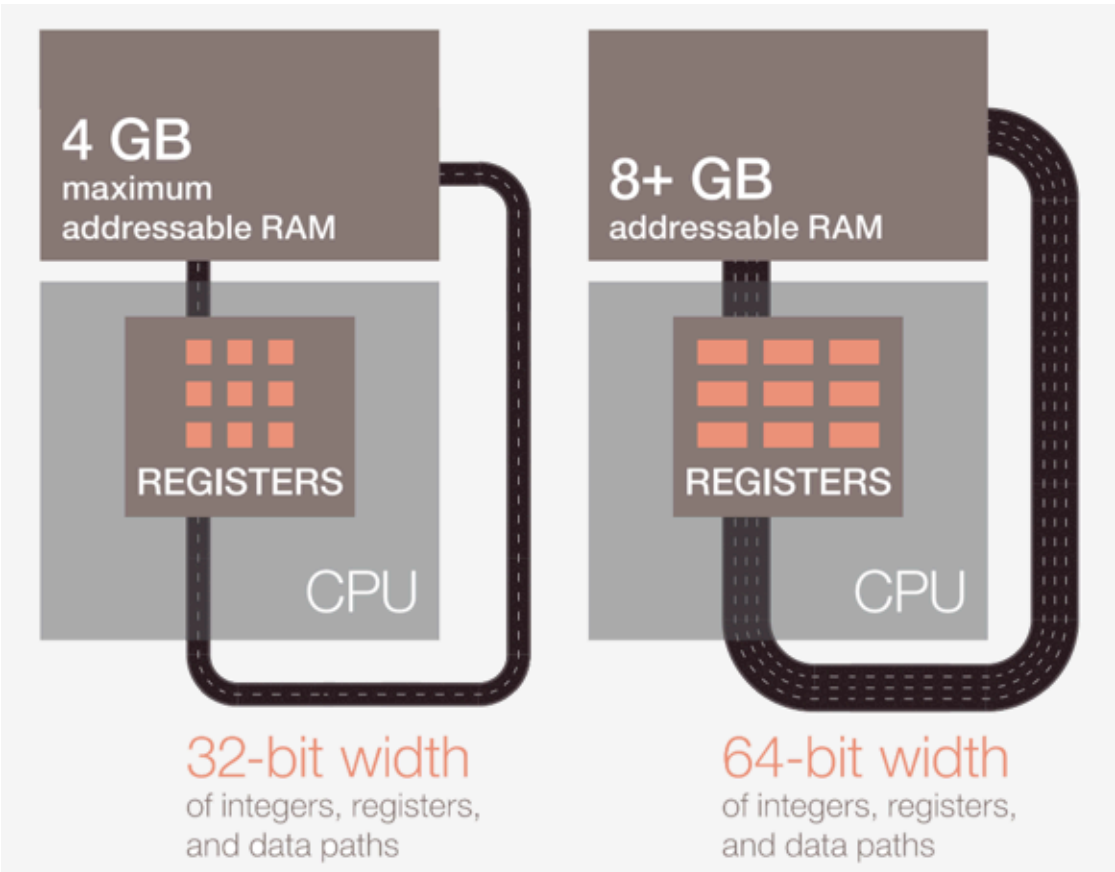
Разные платформы, разные цели

Источник данных

Компьютерная система

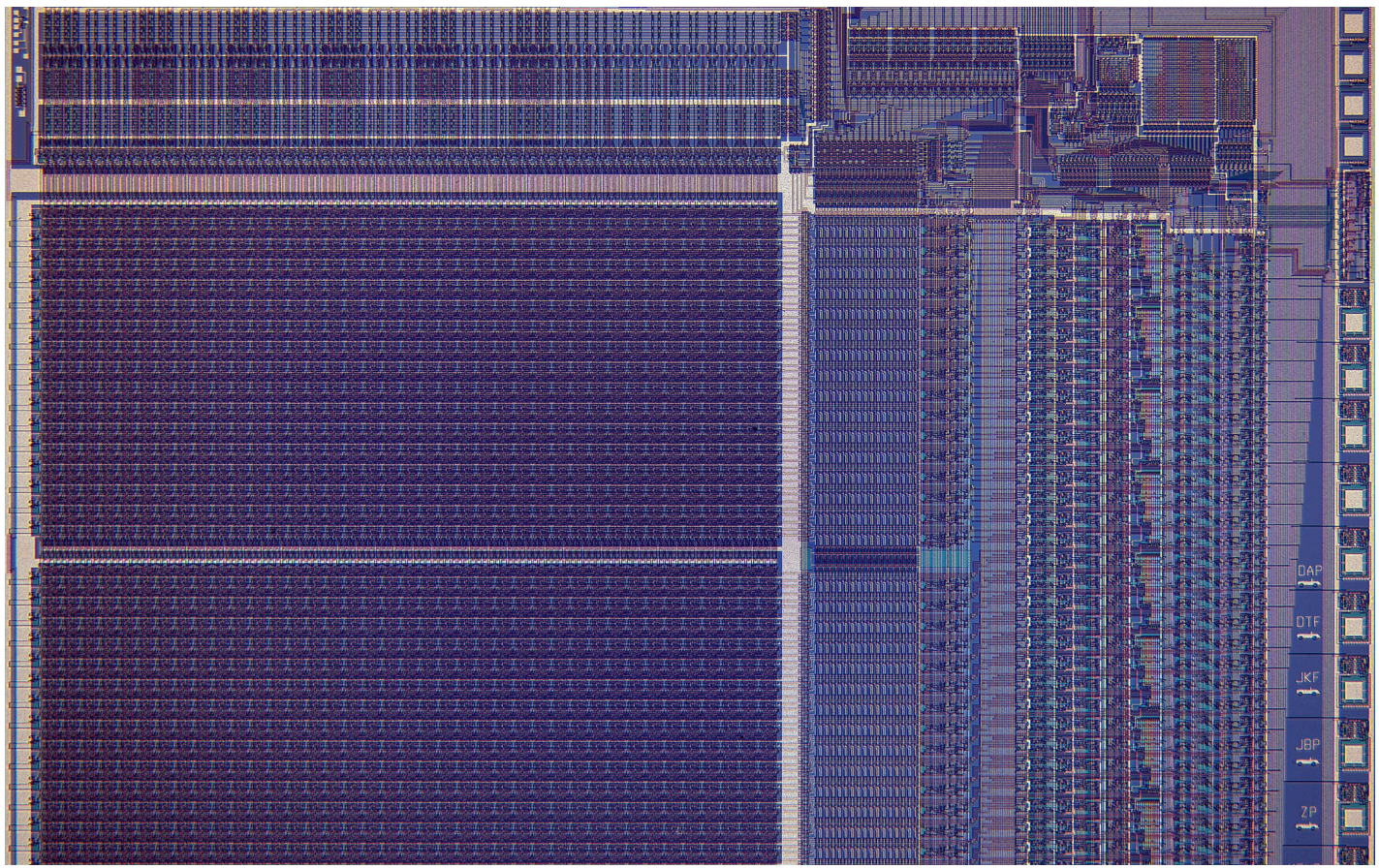
Более подробно

Что значит 32- и 64-битный CPU

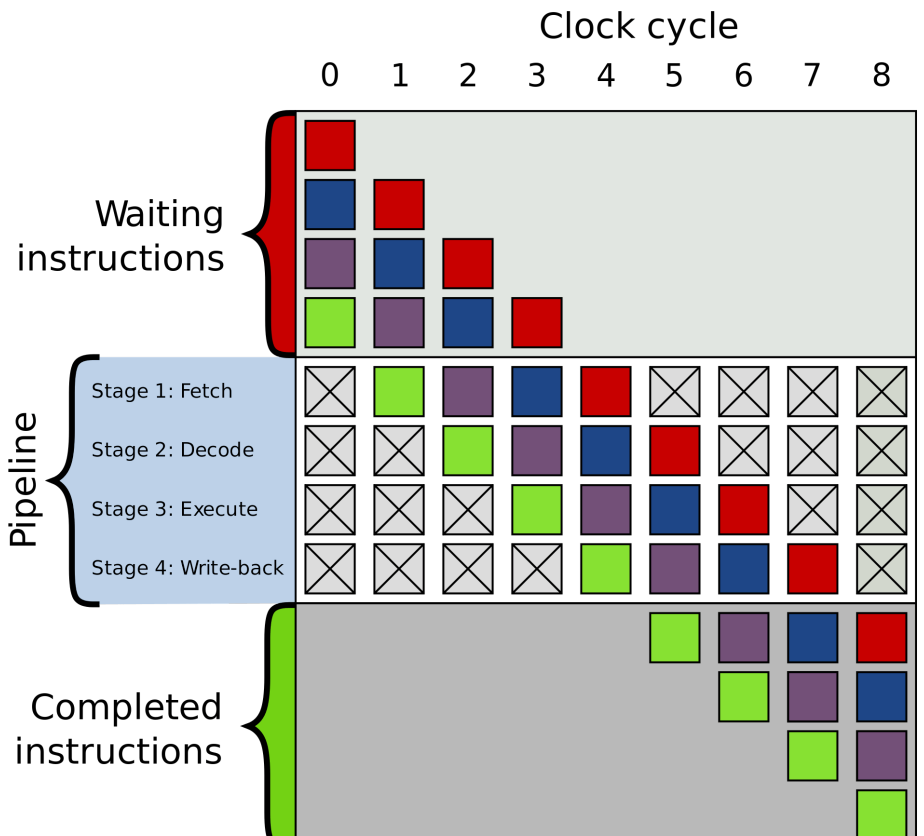


Berkeley RISC I (1981 год)

- Принцип работы: базовый конвейер
- Число транзисторов: ~50 000

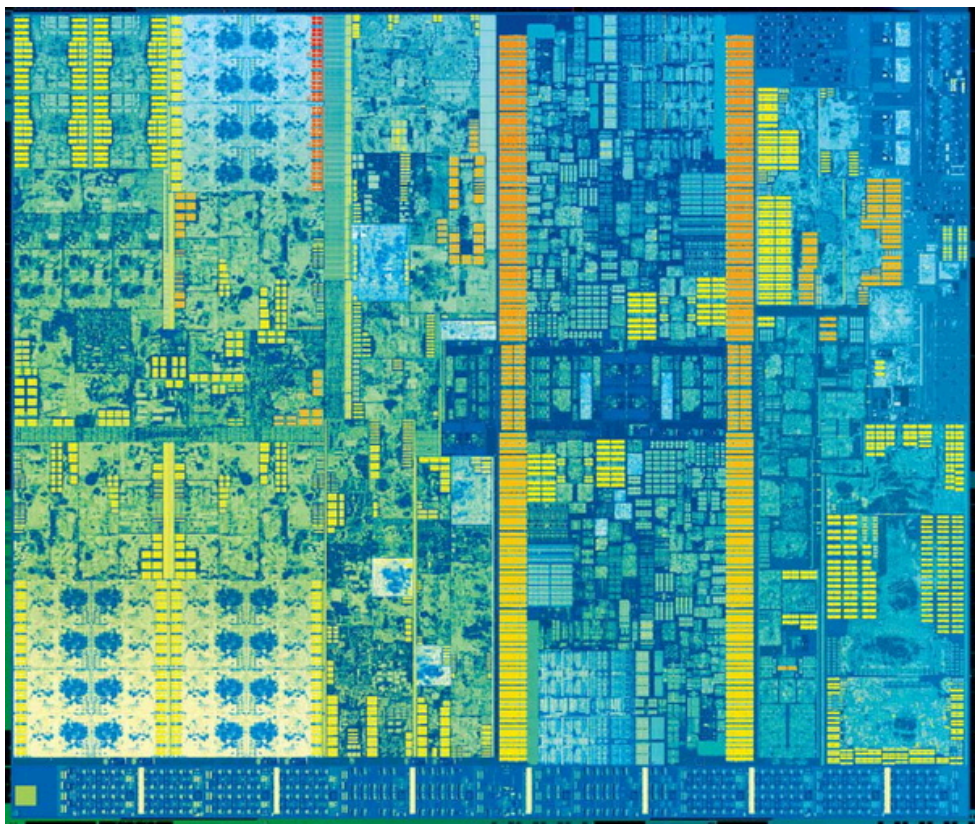


Пример базового конвейера

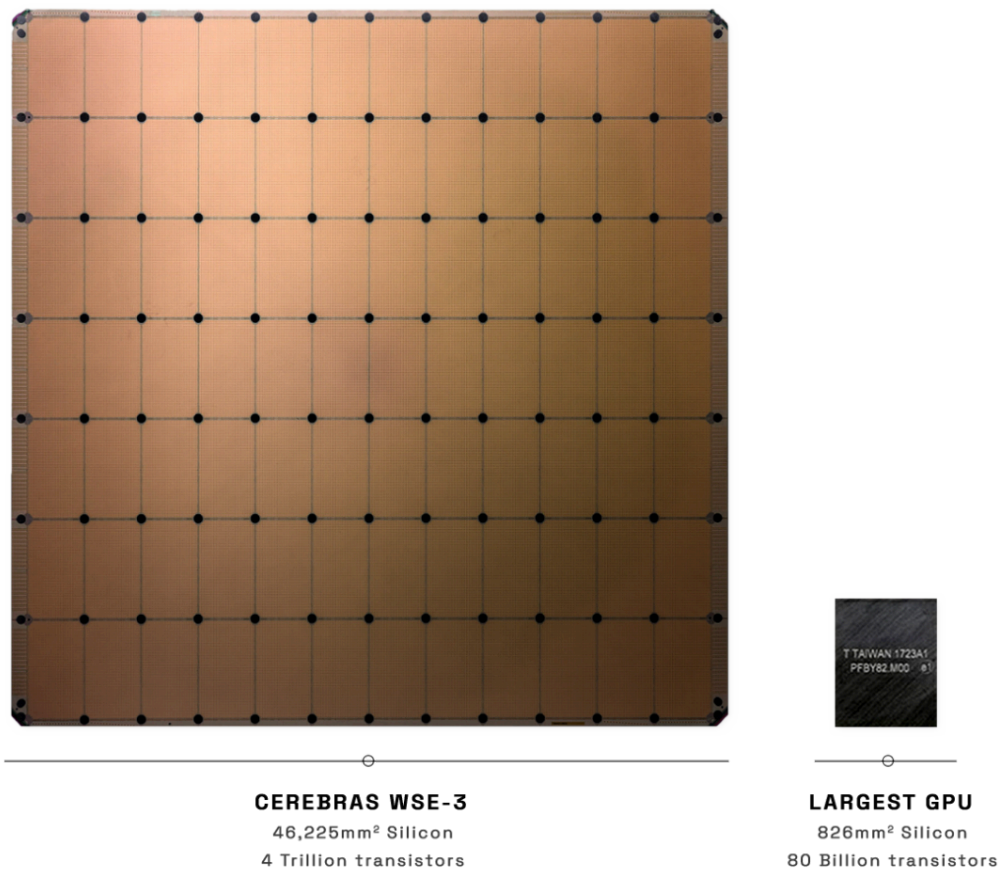


Intel Kaby Lake (2016 год)

- Instruction Level Parallelism
  - суперскалярный
    - SMT
  - VLIW
- Длинный конвейер
  - + исполнение инструкций по мере готовности операндов
- Расширенная система памяти и кэшей
- Data Level Parallelism
  - векторный
  - GPU
- Thread Level Parallelism



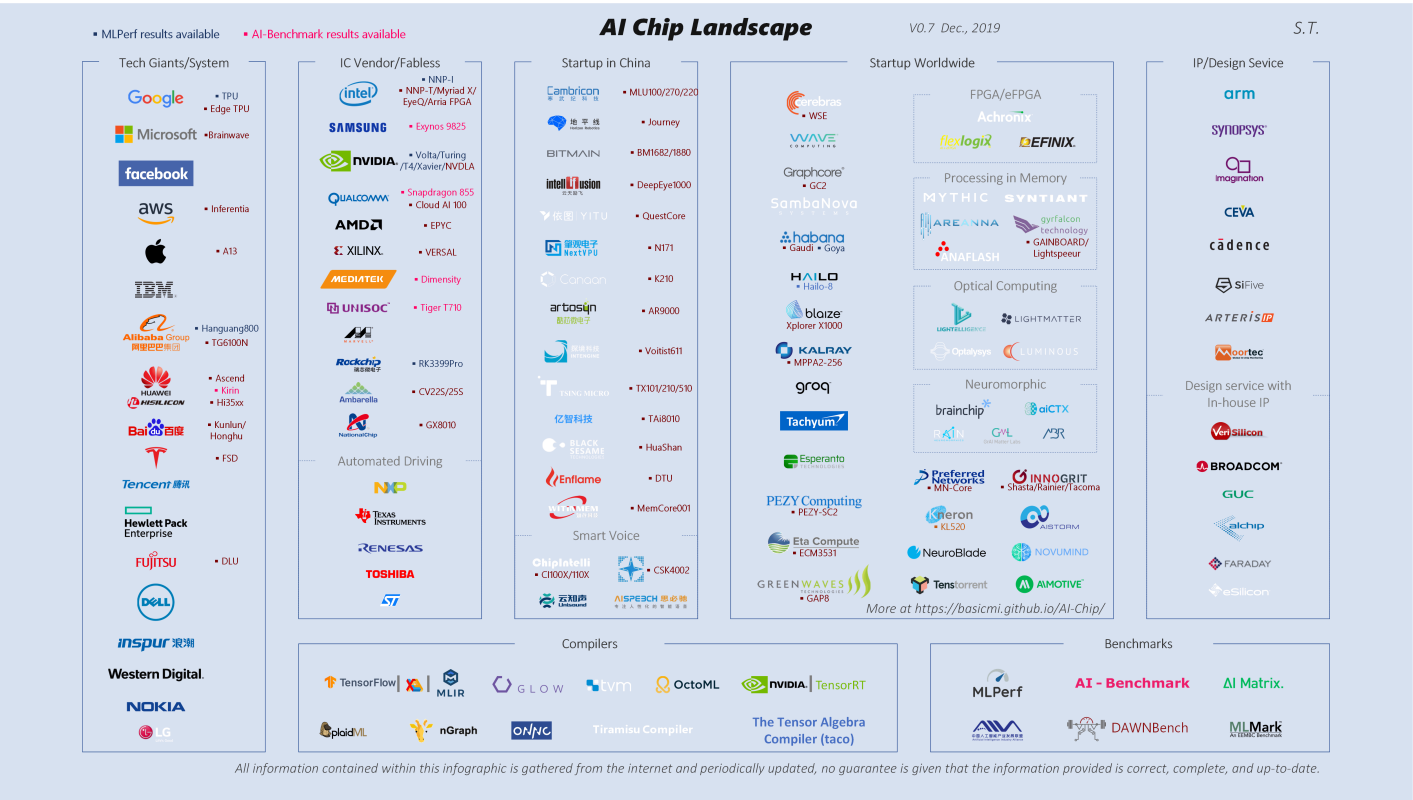
Cerebras's Wafer Scale Engine (2024)



CEREBRAS WSE-3
46,225mm² Silicon
4 Trillion transistors

LARGEST GPU
826mm² Silicon
80 Billion transistors

Производители чипов



Источник

Литература (разделы 1, 2)

- Patterson, D. Computer Organization and Design RISC-V Edition: The Hardware Software Interface / D. Patterson, J. Hennessy. - Morgan Kaufmann, 2020
- **Patterson, D. Computer Architecture: A Quantitative Approach / D. Patterson, J. Hennessy. - Morgan Kaufmann, 2025**
- Computer Architecture by Princeton University

Литература (разделы 1, 2)

- Intel® 64 and IA-32 Architectures Software Developer Manuals
- AMD Tech Docs
- PowerPC Architecture Book
- AnandTech
- iXBT
- THG
- Software optimization resources
- Хабр
- Google

Литература (раздел 3)

- NVIDIA Docs Hub
- Programming Massively Parallel Processors, 4th Edition: A Hands-on Approach
- Параллельные вычисления на GPU. Архитектура и программная модель CUDA: Учебное пособие (А. В. Боресков и др.)

АРХИТЕКТУРА И МИКРОАРХИТЕКТУРА

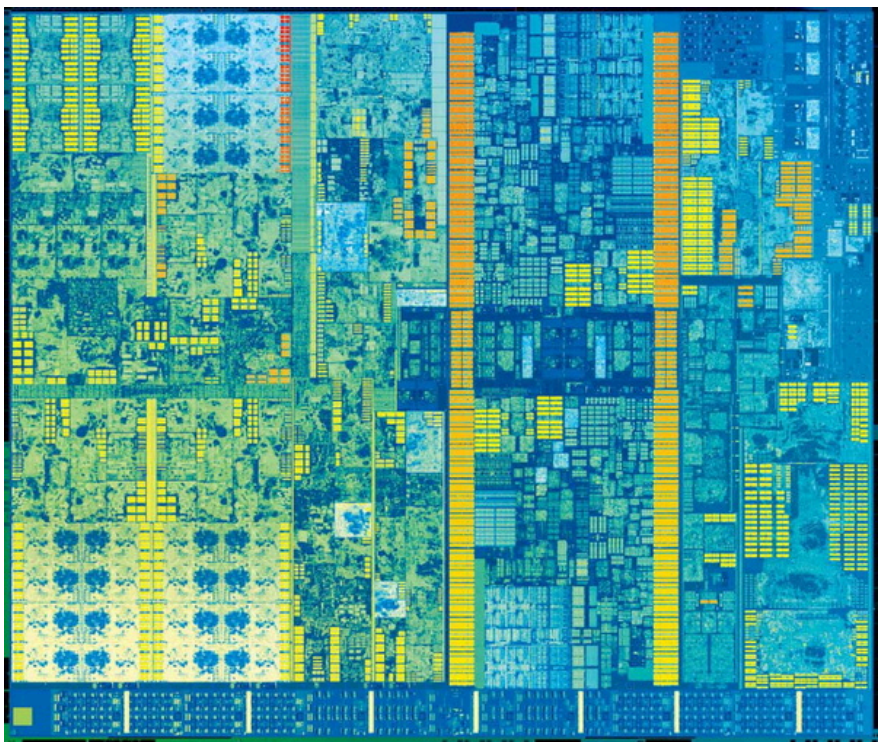
Архитектура vs Микроархитектура

Архитектура vs Микроархитектура

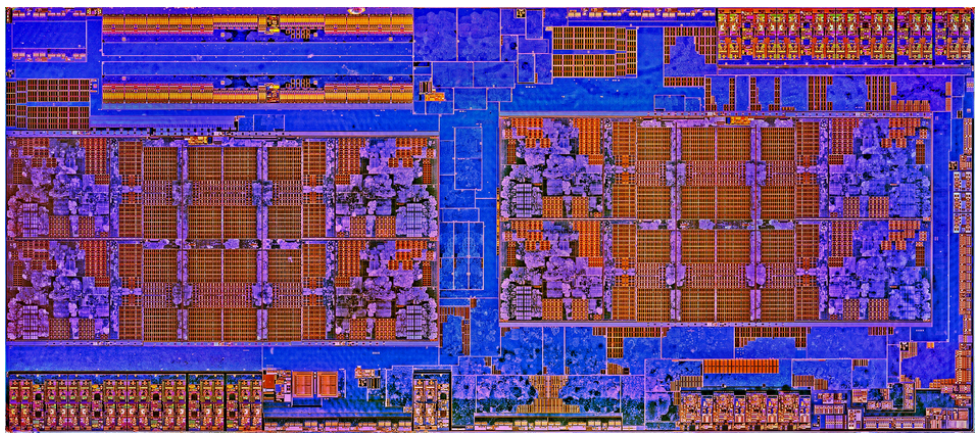
- **Архитектура / Instruction Set Architecture (ISA)**
  - programmer visible state (память и регистры)
  - доступные операции (инструкции и как они работают)
  - семантика исполнения (прерывания)
  - ввод / вывод
  - типы данных и их размеры
- **Микроархитектура / Microarchitecture**
  - исследования, как улучшить ISA по той или иной метрике
  - длина конвейера, размер кэша, набор функциональных блоков и их взаимодействие

Общая архитектура. Разная микроархитектура

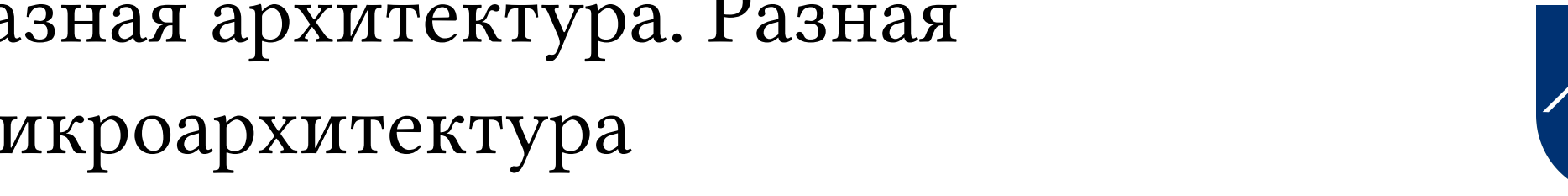
Intel Kaby Lake



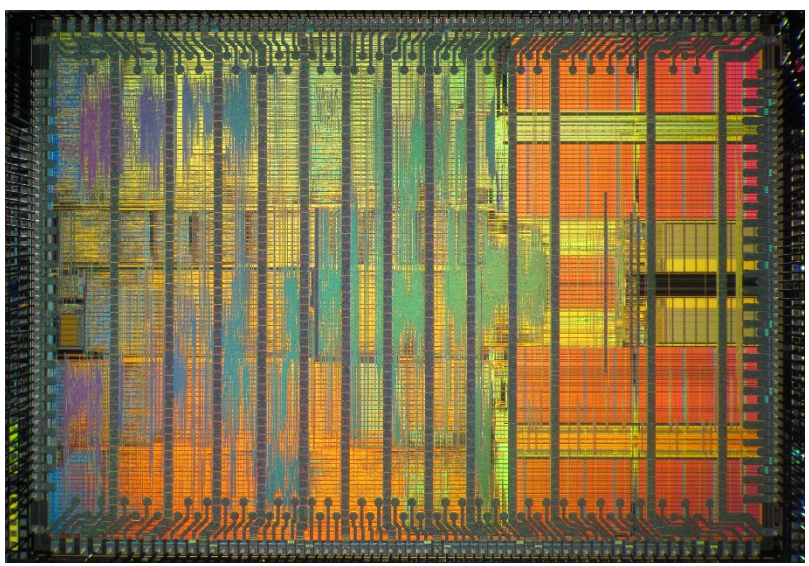
AMD Zen



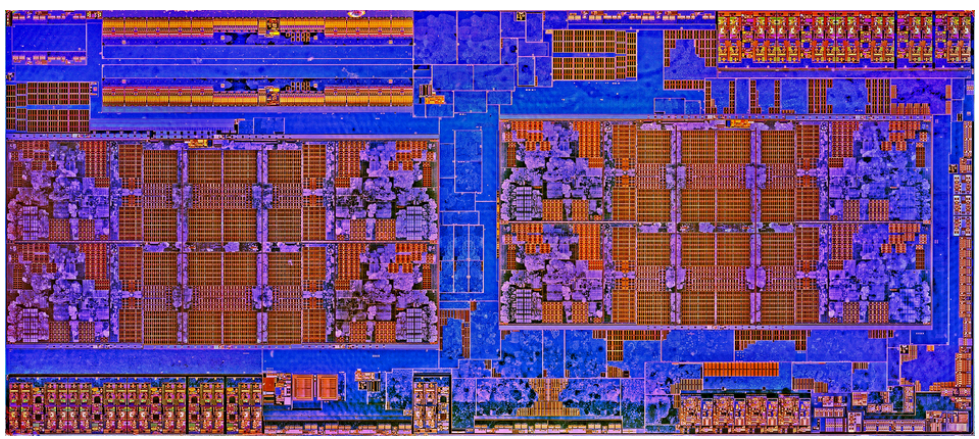
Разная архитектура. Разная микроархитектура



PowerPC 603

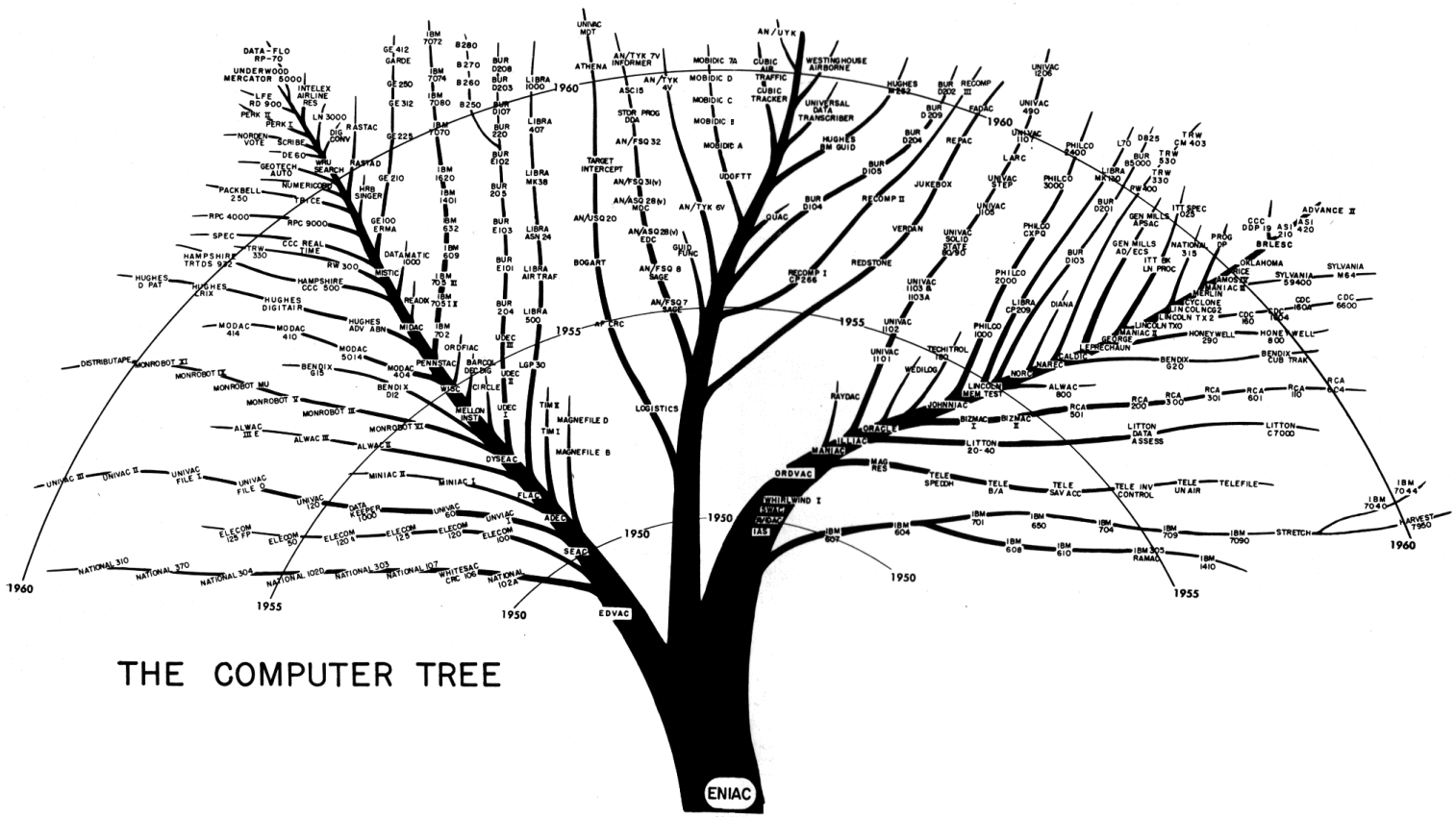


AMD Zen

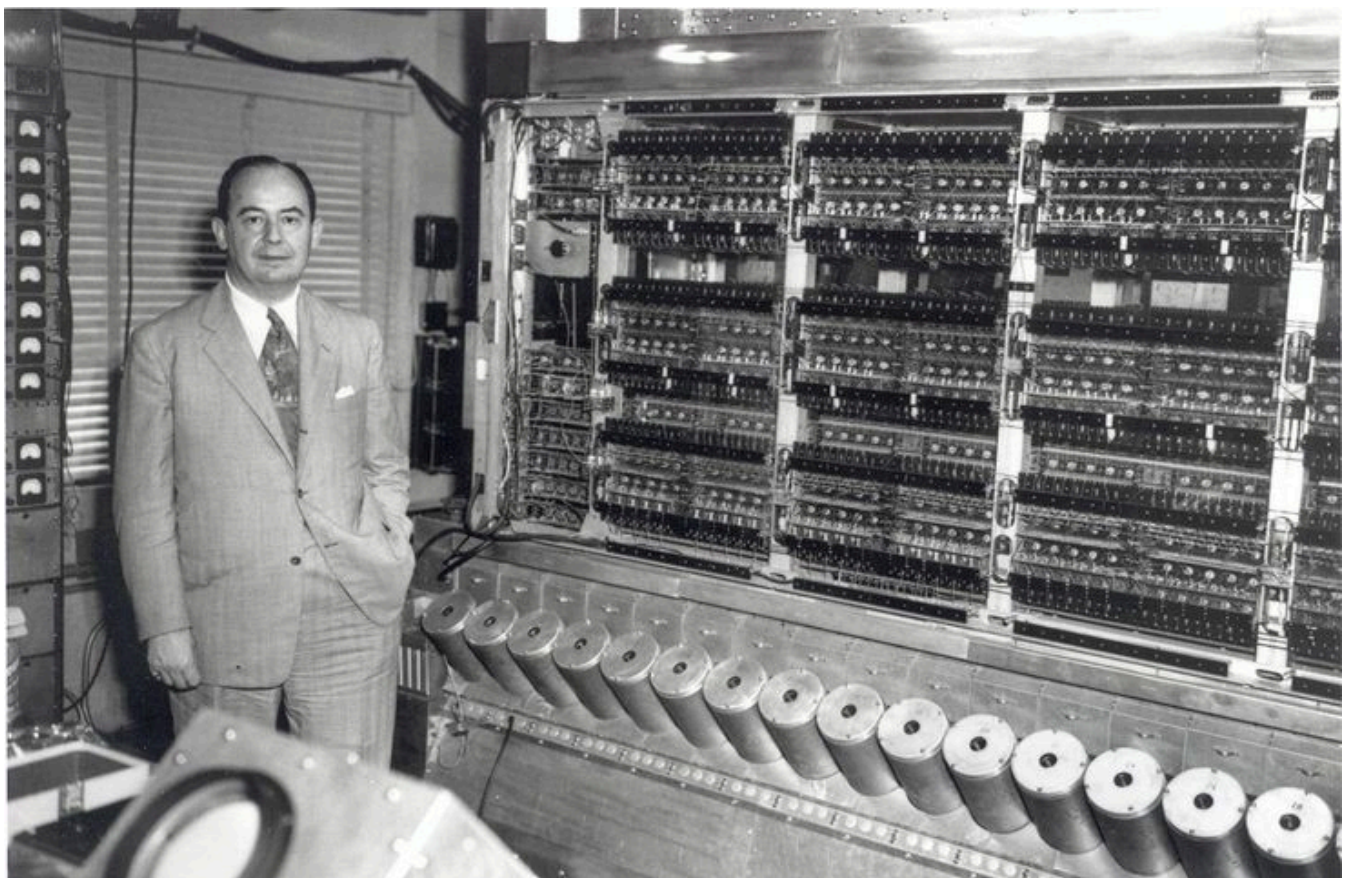


ИСТОРИЧЕСКИЙ ОБЗОР АРХИТЕКТУР. СПОСОБЫ КЛАССИФИКАЦИИ

С чего все начиналось?



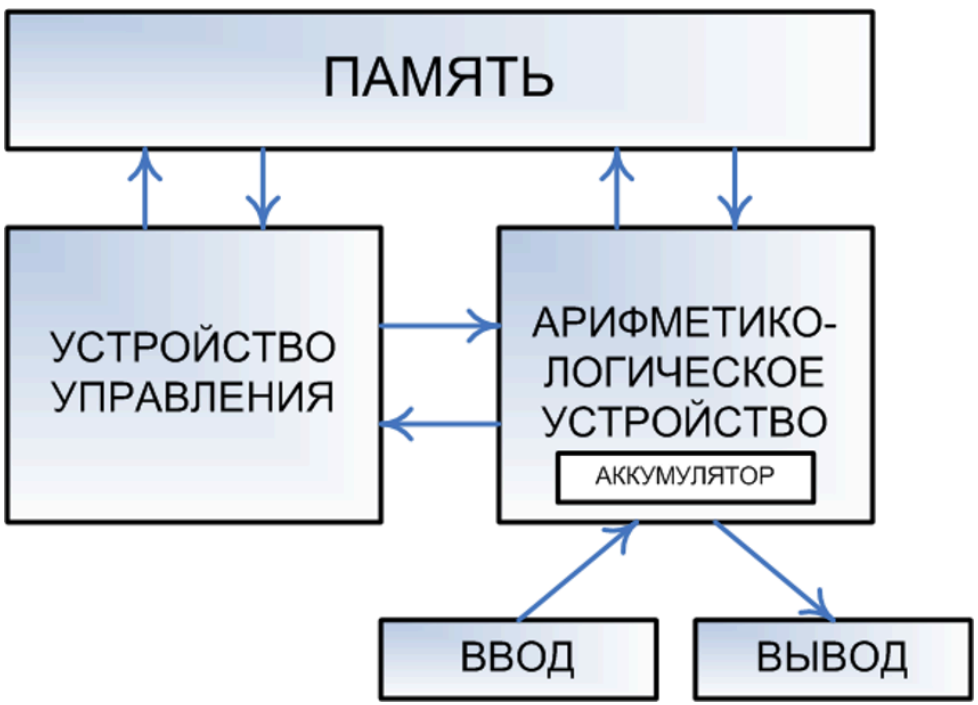
IAS-машина (1952 г.)



Варианты классификации архитектур

- Control Flow / Data Flow архитектуры
- по способу хранения операндов
  - стековая
  - аккумуляторная (IBM 7090, DEC PDP-8, Агат-8)
  - **регистровая**
- по доступу к операндам
- по типу архитектуры: CISC, RISC, VLIW
  - MIPS, PowerPC, SPARC, 80x86, x86-64, ARM, ...

Машины управляемые потоком команд (Архитектура фон Неймана, control flow)



- принцип однородности памяти
- принцип адресности
- принцип программного управления
- принцип двоичного кодирования

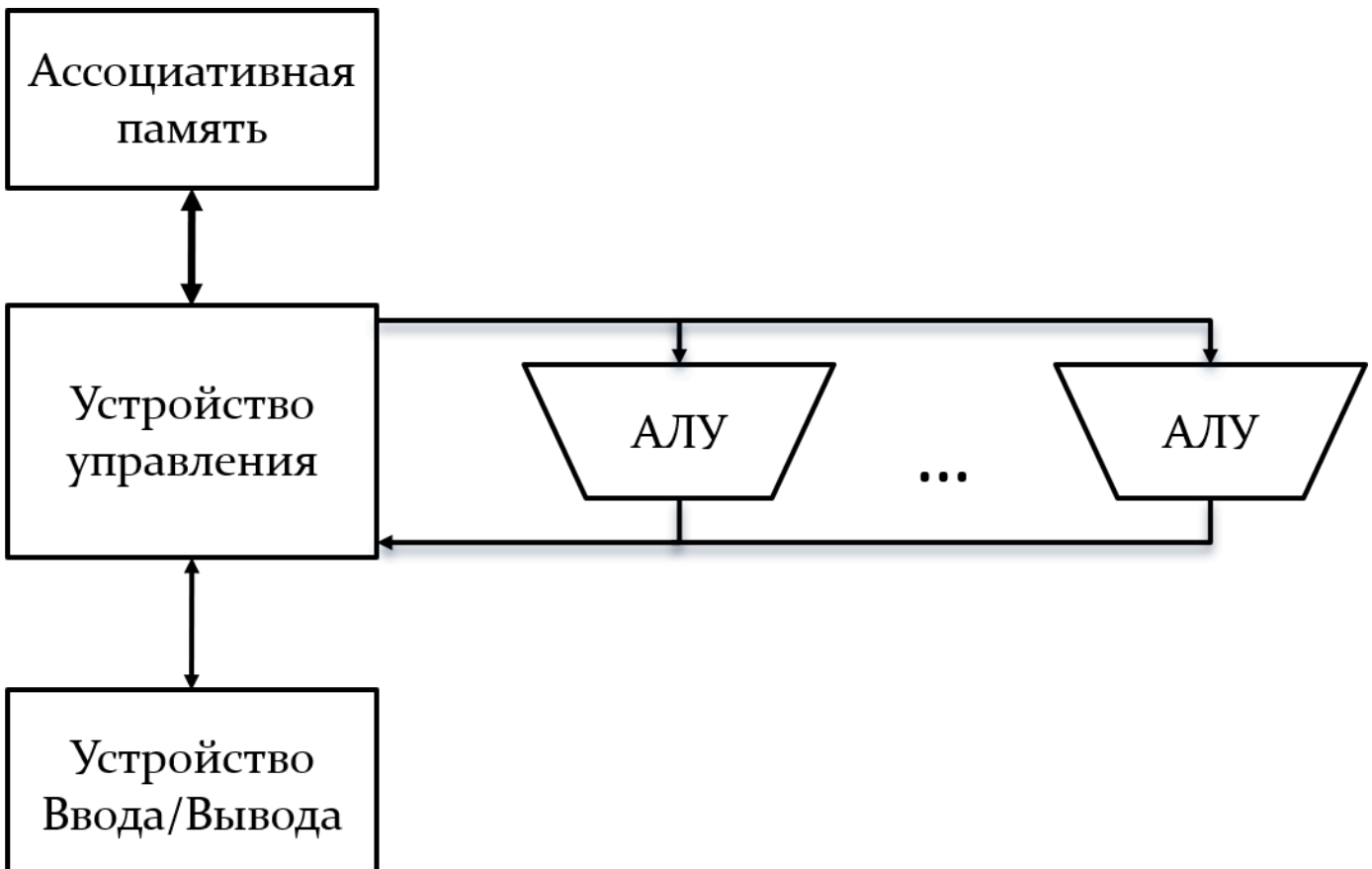
Этапы цикла выполнения инструкций МУПК

- процессор выполняет чтение из памяти текущей инструкции по адресу, хранящемуся в регистре счётчика команд
- декодирование инструкции и выборка ее операндов в АЛУ
- исполнение инструкции на АЛУ
- сохранение результата выполнения инструкции
- изменение счетчика команд в соответствии с типом инструкции. Переход к п.1

Особенности МУПК

- порядок исполнения инструкций задается счетчиком команд и специальными управляющими инструкциями
- данные и инструкции хранятся в общей памяти
  - загрузка/выгрузка регистров
- последовательное исполнение инструкций
  - многопроцессорные/многоядерные системы
    - проблемы памяти
    - отсутствие гарантии корректности данных между потоками

Машины, управляемые потоком данных (Dataflow)



Особенности МУПД

- Параллельная парадигма исполнения инструкций
- Отсутствует счетчик команд
- Инструкции исполняются в порядке готовности их аргументов

Этапы цикла выполнения инструкций МУПД

- Запрос и выборка готовых к исполнению инструкций в ассоциативной памяти. Количество выбираемых инструкций меньше или равно числу АЛУ.
- Исполнение инструкций на АЛУ.
- Сохранение результатов выполнения инструкций в память. Переход к пункту 1.

Машины, управляемые потоком данных (Dataflow)

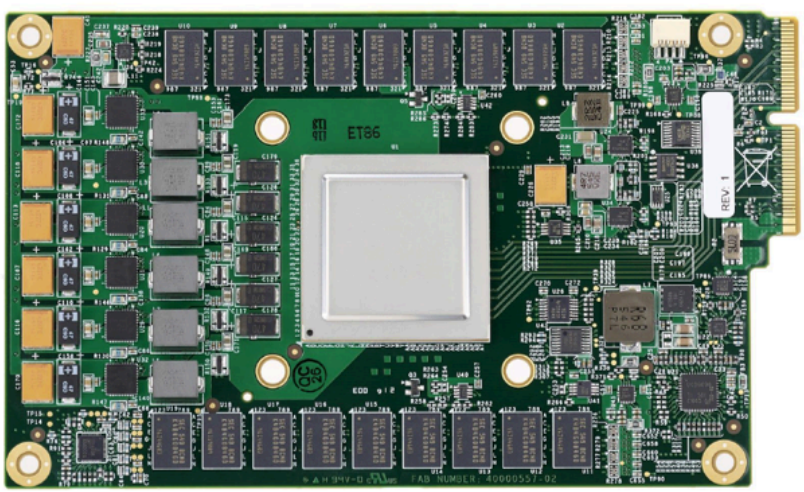


Figure 3. TPU Printed Circuit Board. It can be inserted in the slot for an SATA disk in a server, but the card uses PCIe Gen3 x16.

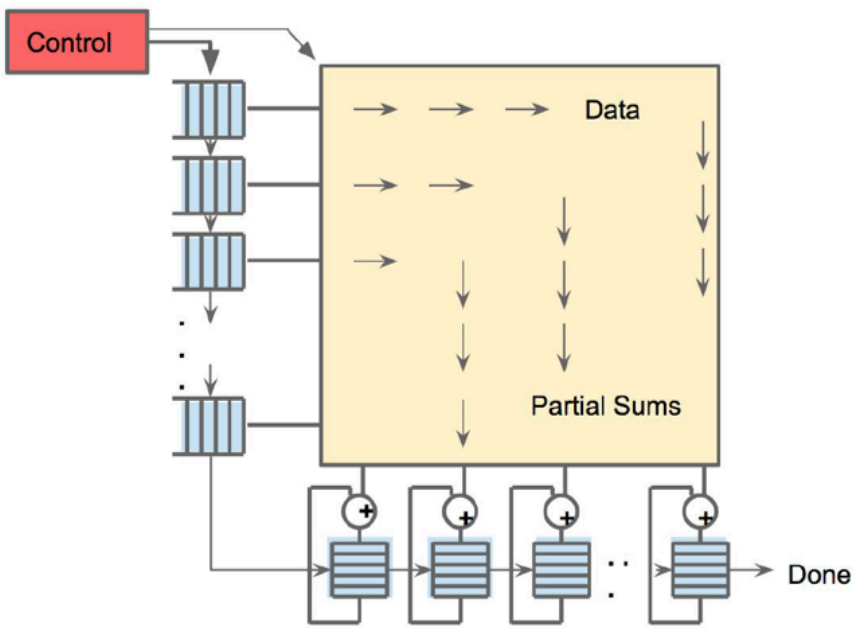
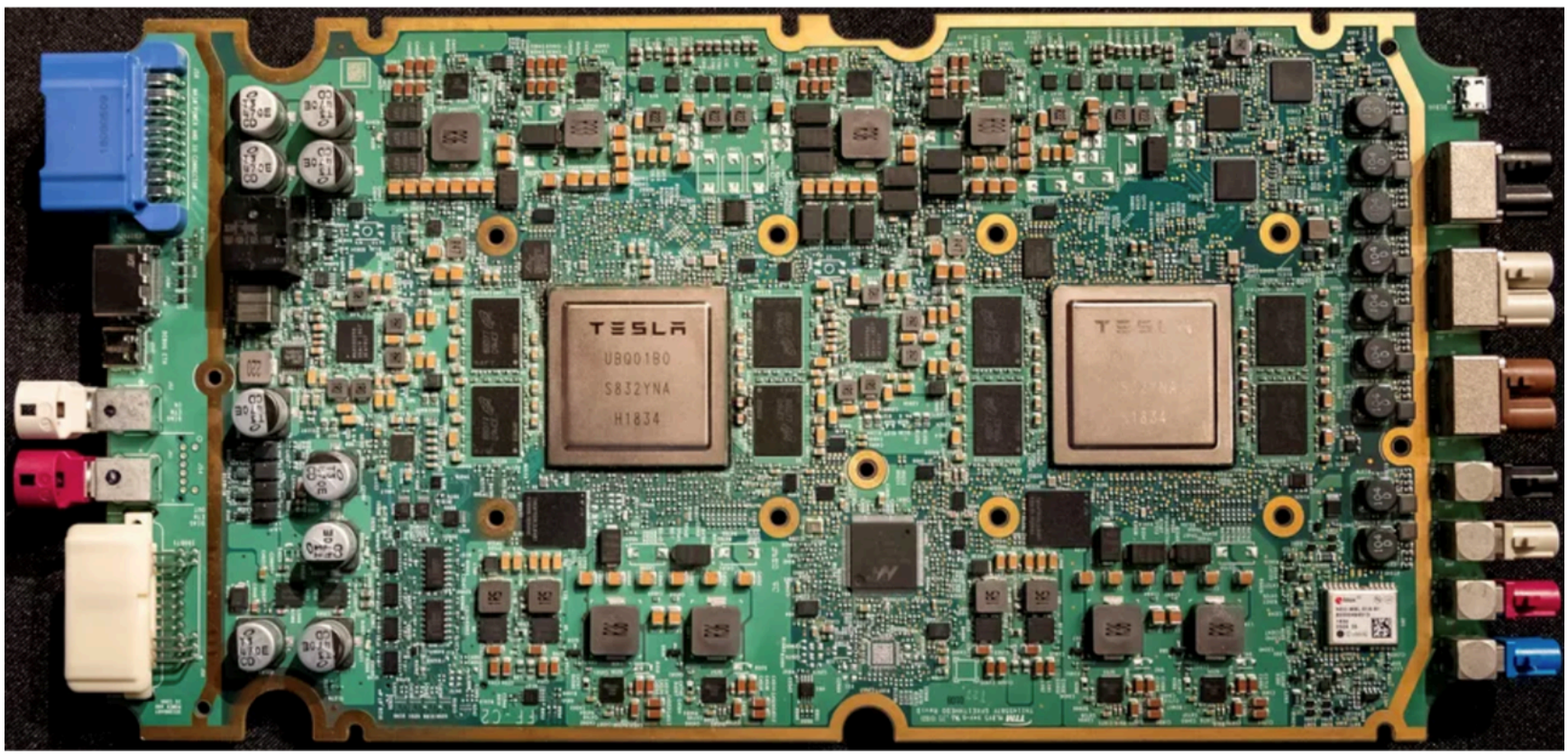


Figure 4. Systolic data flow of the Matrix Multiply Unit. Software has the illusion that each 256B input is read at once, and they instantly update one location of each of 256 accumulator RAMs.

Dataflow-архитектуры

- ML-ускоритель: 260 mm^2, 6 billion transistors, 600 GFLOPS GPU, 12 ARM 2.2 GHz CPUs.
- Two redundant chips for better safety.


TESLA
UBQ01B0
S8J2YNA
H1834

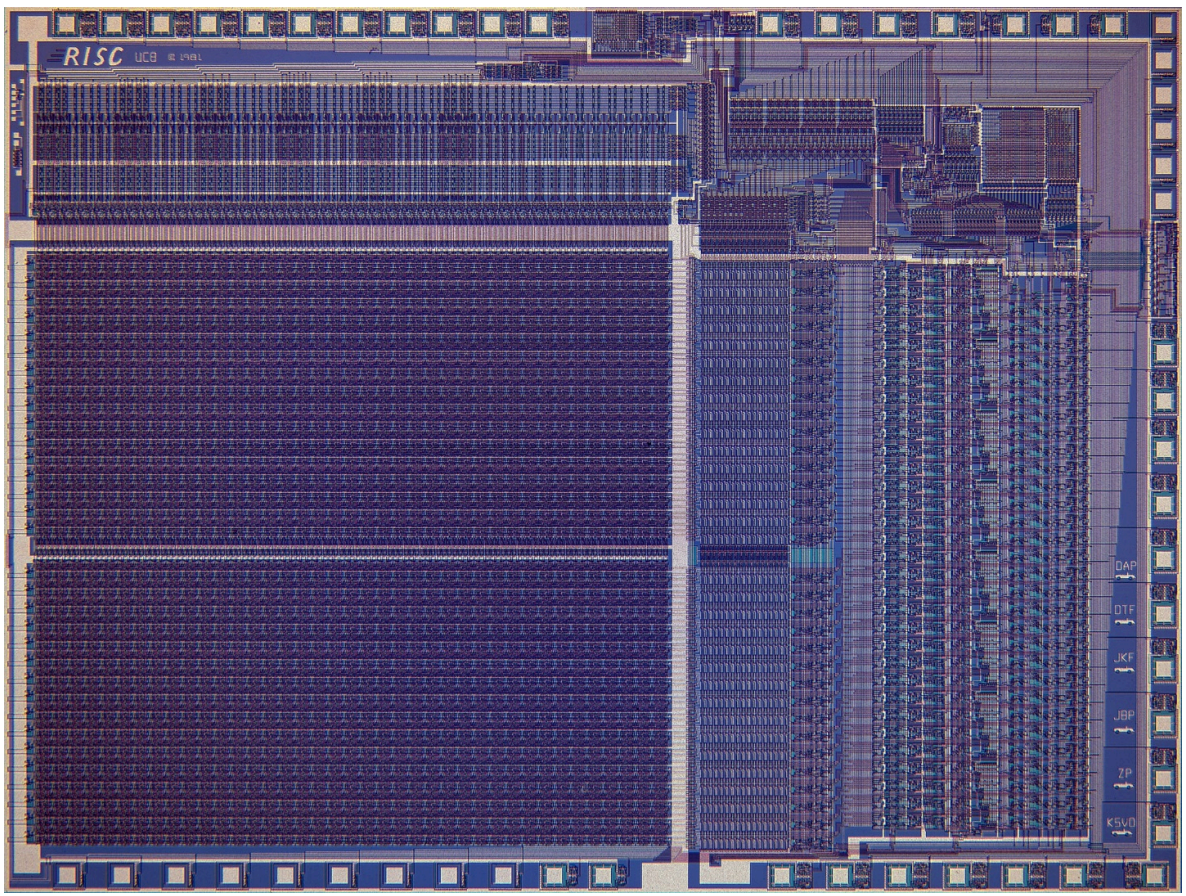
Классификация по доступу к операндам

- **регистр - регистр (0)**
- **регистр - память (1)**
- память - память (2+)
- стекируемая
- аккумуляторная

CISC архитектура (Complex Instruction Set Computer)

- малое количество регистров
- большое количество режимов адресации
- сложные инструкции
- переменная длина инструкции

Berkeley RISC I (1980-1982 годы)



RISC архитектура (Reduced Instruction Set Computer)

- архитектура загрузки-сохранения (load-store)
- большое количество универсальных регистров (register-register)
- простые инструкции
- ограниченное число режимов адресации
- инструкции фиксированной длины

Причины перехода от CISC к RISC

- конвейеризация процессора
- увеличение объема памяти
- совершенствование компилятора

VLIW
(Very Long Instruction Word)

- *более подробно в отдельной лекции*
- архитектура, специально разработанная для повышения количества одновременно исполняемых инструкций
- позволяет создать более простой процессор, чем RISC архитектура

Ключевая идея VLIW

- дано: конкретный CPU с фиксированным числом функциональных устройств (ФУ)
- принцип работы:
  - инструкция — пакет простых подинструкций (их кол-во = число ФУ)
  - пакет обрабатывается ЦЕЛИКОМ по одному за такт

CISC → RISC → VLIW

Обзор архитектур

- MIPS (умерло)
- SPARC (умирает)
- Power (все сложно)
- 80x86 (все сложно)
- ARM
- RISC-V
- … (за прошедшие 20 лет — не менее 3 прекратили существование и несколько зарождается)
  - что будет еще через 10 лет???

Архитектура MIPS (Microprocessor without Interlocked Pipeline Stages)

- ISA: RISC, MIPS32/64 Release 6 (2014)
- область применения
  - встраиваемые решения
    - для 1981 года — **высокопроизводительные**
- начало разработки — 1981 год
  - 1984 — MIPS Technologies — John L. Hennessy (Stanford)
  - 2018 — Wave Computing
- март 2021 г. — переход на RISC-V

Технические требования

- совершенствование конвейера:
  - согласовать каждую подфазу каждой команды (в т.ч. кэширование) в один цикл
    - избегание необходимости в блокировках между стадиями
    - пропуск на конвейере только один цикл
    - цена: исключение «тяжелых» операций
  - исключение регистрового окна

Характеристика архитектуры

- малое количество простых инструкций
- всего 3 формата инструкций
- широкое распространение в академических кругах

Представители архитектуры

- Cisco, Linksys, ZyXEL и MikroTik
- кабельные и ADSL-модемы
- смарт-карты, механизмы лазерных принтеров, цифровые приставки
- роботы, карманные компьютеры и т.д.

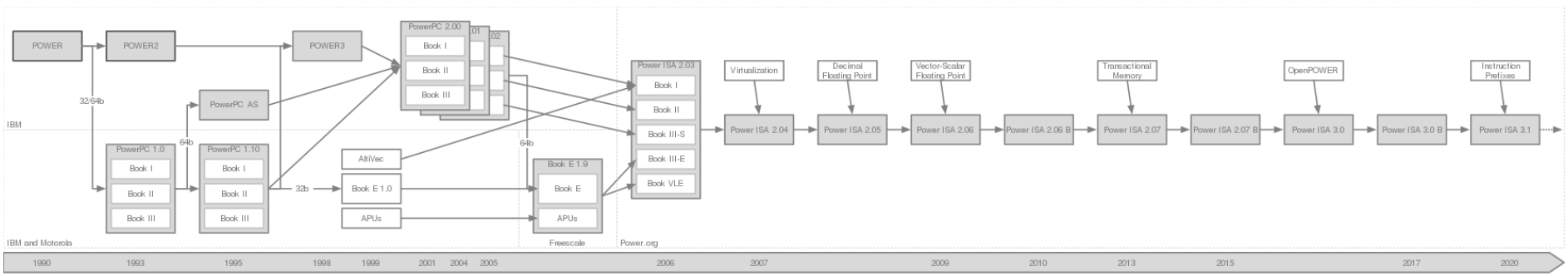
Необычные представители

ASCI Blue Mountain (1998-2004)

- Los Alamos National Laboratory
- 6,144 MIPS R10000
- теоретическая пиковая производительность: 3.072 teraflops

PowerPC (Performance Computing)

- архитектура POWER
  - Performance Optimization With Enhanced RISC
- область применения
  - высокопроизводительные серверные, настольные, встраиваемые решения
- 1991 г.: альянс AIM: Apple, IBM, Motorola
- 2013 г.: OpenPOWER Foundation


POWER
POWER2
POWER3
32/64b
PowerPC AS
IBM
64b
PowerPC 1.0
Book I
Book II
Book III
PowerPC 1.10
Book I
Book II
Book III
32b
AltiVec
Book E 1.0
APUs
PowerPC 2.00
Book I
Book II
Book III
64b
Book E 1.9
Book E
APUs
IBM and Motorola
Freescale
Power ISA 2.03
Book I
Book II
Book III-S
Book III-E
Book VLE
Power.org
Virtualization
Power ISA 2.04
Decimal Floating Point
Power ISA 2.05
Vector-Scalar Floating Point
Power ISA 2.06
Power ISA 2.06 B
Transactional Memory
Power ISA 2.07
Power ISA 2.07 B
OpenPOWER
Power ISA 3.0
Power ISA 3.0 B
Instruction Prefixes
Power ISA 3.1
1990
1993
1995
1998
1999
2001
2004
2005
2006
2007
2009
2010
2013
2015
2017
2020

IBM RISC System/6000 (RS/6000)

- выпускался: 1990-2002 годы
- охватывал сегменты: desktop, server, workstation, supercomputer

Наиболее известные представители

Наиболее известные представители

- компьютеры фирмы Apple (с 1994 по 2005 годы)
  - альянс AIM (Apple Computer, IBM, Motorola)
- X-BOX 360 (2005 г., ядро Xenon)
- Sony PlayStation 3 (2006 г., ядро Cell)

Summit

- Введен в эксплуатацию: 06.2018
- 5 место в TOP-500 (ноябрь 2022 г.)
- МинЭнергетики США
- 9216 POWER9 22-core CPUs
- 27 648 Nvidia Tesla V100
- Пиковая производительность:
  - 200,795 PFlop/s
  - 10 096 kW

SPARC (Scalable Processor ARChitecture)

- ISA: IEEE 1754-1994
- Область применения:
  - высокопроизводительные серверные, настольные решения
- Основные производители процессоров:
  - Sun Microsystem, Fujitsu
- В 1986 публикуется первая версия архитектуры SPARC Version 7 (V7)
- В 1990 выходит SPARC Version 8
- В 1995-1996 выходит 64-х битная версия SPARC Version 9, процессоры Sun Ultra SPARC I, Fujitsu SPARC64

OpenSPARC

- Офиц.сайт
- 2005 г.: UltraSparc T1
- 2007 г.: UltraSparc T2
- 2020 г.: Oracle SPARC M8

SORA-MA

- Национальное аэрокосмическое агентство
- 99 место в TOP500 (ноябрь 2020 г.)
- SPARC64 XIfx 32C / 1.6 МВт / 3,481.1 TFlops

Представители архитектуры

- Fujitsu SPARC64 XII (2017, серверы SPARC M12)
- Fujitsu SPARC64 XIfx (2015, суперкомпьютер PRIMEHPC FX100)
- Oracle SPARC M8 (сентябрь 2017, серверы)

80x86

- Область применения:
  - высокопроизводительные настольные решения, сервера среднего и начального уровня, суперкомпьютеры (с 2002)
- Основные производители процессоров:
  - Intel, AMD, много мелких компаний из Китая
- В 1978 выпущен 16-битный процессор 8086
- Архитектура 80x86 стала дальнейшим развитием архитектуры успешного 8-ми битного процессора Intel 8080
- В 1980 году была разработана архитектура вещественного сопроцессора x87

80x86 (продолжение)

- В 1985 году вместе с выпуском процессора Intel 80386 вышла 32-х битная версия архитектуры:
  - страничная адресация
  - защищенный режим
- 1997 — расширение MMX (57 инструкций)
- 1999 — расширение SSE (70 инструкций)
- 2001 — расширение SSE2 (144 инструкций)
- 2003 — AMD Opteron с 64-х битной версией x86-64 (EM64T)

80x86 (окончание)

- 2003 — расширение SSE3 (13 инструкций)
- 2006 — расширение SSSE3 (16 инструкций)
- 2007 — расширение SSE4.1 (47 инструкций)
- 2008 — расширение SSE4.2 (8 инструкций)
- 2011 — расширение AVX
- 2013 — расширение AVX2
- 2016 — расширение AVX-512

ARM (Advanced RISC Machine)

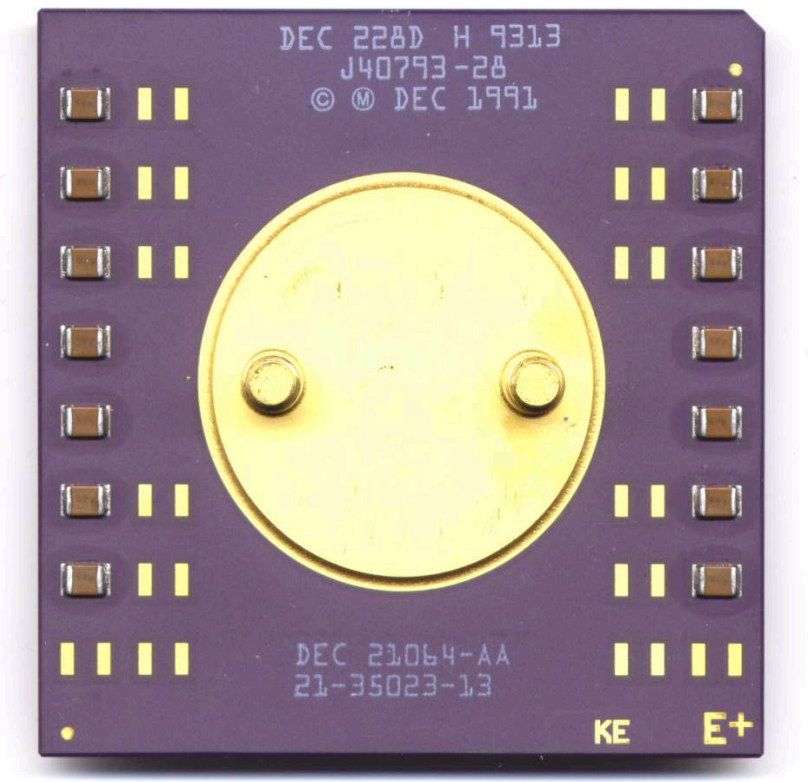
- Область применения:
  - встраиваемые решения (смартфоны, планшеты)
- Разработчик:
  - ARM Limited
- Разработка: с 1985 г. (ARM1)
- Семейства:
  - ARM7, ARM9, ARM11, Cortex, Neoverse

RISC архитектуры, закончившие свое существование

- Alpha
- PA-RISC
- IA-64

DEC Alpha

- Область применения:
  - высокопроизводительные сервера и суперкомпьютеры
- Основные производители процессоров:
  - DEC → Compaq (2001) → HP (до 2007 г.)
- Все процессоры 64-х битные
- С 1992 по 2001 год процессоры Alpha являлись самыми быстрыми в мире
- В 1992 выпущен первый процессор Alpha 21064



DEC Alpha

- Она могла стать Первой: история процессорной архитектуры Alpha
  - часть 1
  - часть 2
  - часть 3
  - часть 4

PA-RISC (Precision Architecture)

- область применения:
  - высокопроизводительные сервера и суперкомпьютеры
  - наиболее сложной и развитой системой команд среди всех RISC архитектур
- основные производители процессоров:
  - HP
- в 1989 выпущен первый 32-х битный процессор PA-7000
- в 1996 выпущен 64-х битный процессор PA-8000
- системы сняты с продаж в 2003 году, после перехода HP на процессоры Intel Itanium 2

IA-64

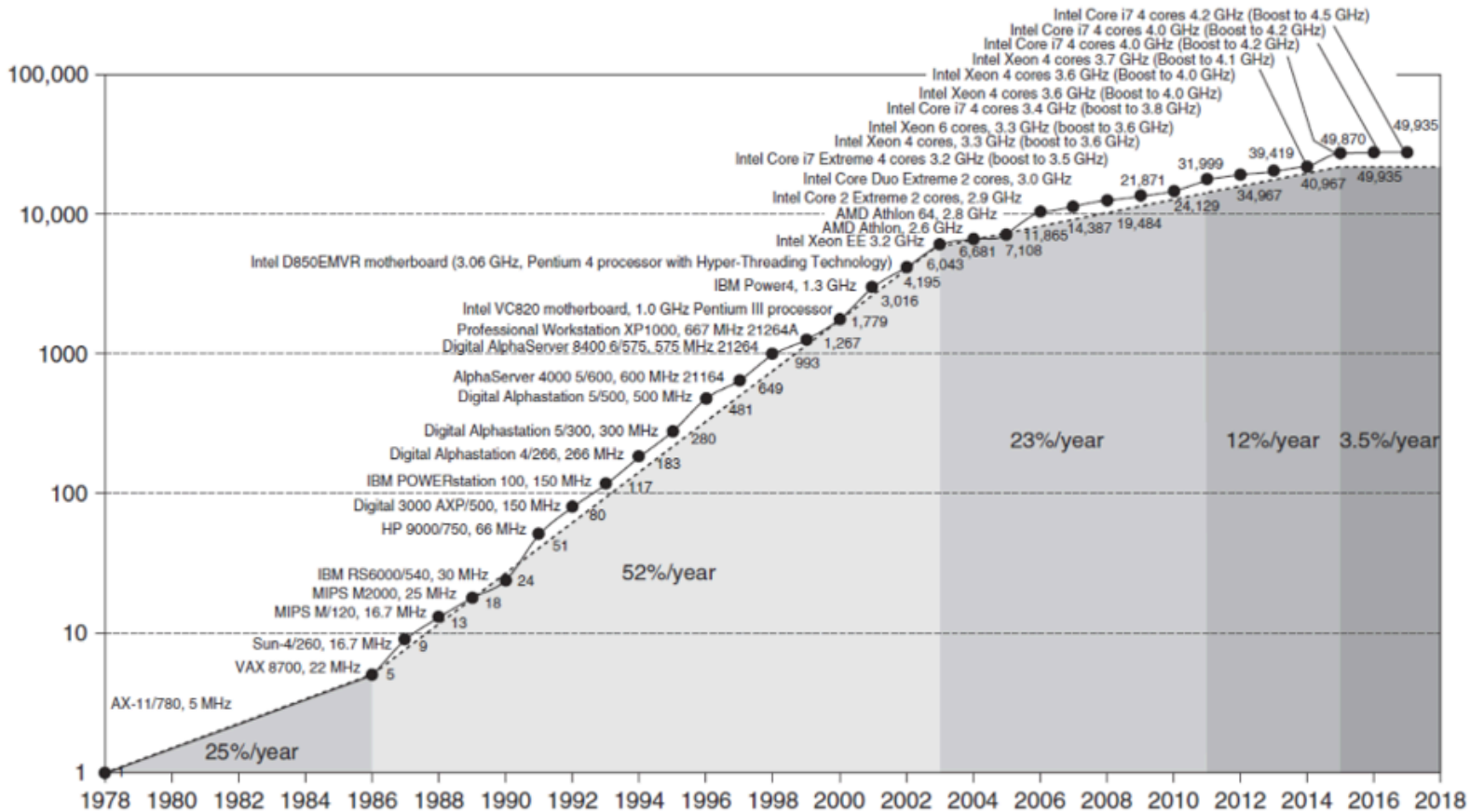
- область применения:
  - высокопроизводительные сервера и суперкомпьютеры
- разработка:
  - Intel + HP (с 1994 г.)
- архитектура относится к новому поколению EPIC
- ядра:
  - Intel Itanium (2001 г.)
  - Intel Itanium 2 (2002-2009 г.)
  - Itanium 9700 (2017 г.)

ТРЕНДЫ ПРИ ПРОЕКТИРОВАНИИ ВЫЧИСЛИТЕЛЬНЫХ СИСТЕМ

Тренды в технологиях

- улучшение производительности:
  - совершенствование полупроводниковых технологий
    - размер элементов, тактовая частота
  - совершенствование компьютерных архитектур
    - высокоуровневые языки программирования
    - переход на RISC-архитектуры
  - совместные усилия позволили:
    - «легкие» компьютеры
    - управляемые/интерпретируемые языки программирования, основанные на производительности

Рост производительности



Тренды в технологиях

- невозможно продолжать использовать параллелизм на уровне инструкций (Instruction-Level Parallelism (ILP))
  - повышение производительности одного процессора закончилось в 2003 году
- новые модели для повышения производительности:
  - Data-level parallelism (DLP)
  - Thread-level parallelism (TLP)
  - Request-level parallelism (RLP)
- требуется явная реструктуризация приложения

Классы компьютеров

- персональное мобильное устройство (PMD)
  - смартфоны, планшетные компьютеры
  - упор на энергоэффективность и работу в режиме реального времени
- настольные компьютеры
  - акцент на соотношении цены и качества
- серверы
  - упор на доступность, масштабируемость, пропускную способность
- кластеры / компьютеры масштаба склада (Warehouse Scale Computers)
  - используется для «Software as a Service (SaaS)»
  - акцент на доступность и соотношение цены и качества
  - подкласс: суперкомпьютеры, акцент: производительность с плавающей запятой и быстрые внутренние сети

Классы компьютеров

- Internet of Things / Embedded Computers
  - акцент: цена

Параллелизм

- классы параллелизма в приложениях:
  - Data-Level Parallelism (DLP)
  - Task-Level Parallelism (TLP)
- классы архитектурного параллелизма:
  - Instruction-Level Parallelism (ILP)
  - Vector architectures / Graphic Processor Units (GPUs)
  - Thread-Level Parallelism
  - Request-level parallelism (RLP)

Таксономия Флинна

- Single instruction stream, single data stream (SISD)
- Single instruction stream, multiple data streams (SIMD)
  - векторные архитектуры
  - мультимедийные расширения
  - графические процессоры
- Multiple instruction streams, single data stream (MISD)
  - нет коммерческой реализации
- Multiple instruction streams, multiple data streams (MIMD)
  - MIMD с плотной связью
  - MIMD со слабой связью

Определение архитектуры компьютера

- «Старый» взгляд на компьютерную архитектуру:
  - Instruction Set Architecture (ISA)
  - то есть решения, касающиеся:
    - регистры, адресация памяти, режимы адресации, операнды инструкций, доступные операции, инструкции потока управления, кодировка инструкций
- «Реальная» архитектура компьютера:
  - особые требования к целевой машине
  - проектирование для достижения максимальной производительности в рамках ограничений: стоимость, мощность и доступность
  - включает в себя ISA, микроархитектуру, аппаратное обеспечение

Тенденции развития технологий

- технология интегральных логических схем (закон Мура)
  - плотность транзисторов: 35%/год
  - размер подложки (die): 10-20%/год
  - общая интеграция: 40-55%/год
- объем памяти DRAM: 25-40%/год (замедление)
  - 8 Gb (2014), 16 Gb (2019), possibly no 32 Gb
- емкость флеш-памяти: 50-60%/год
  - в 8-10 раз дешевле/бит, чем DRAM

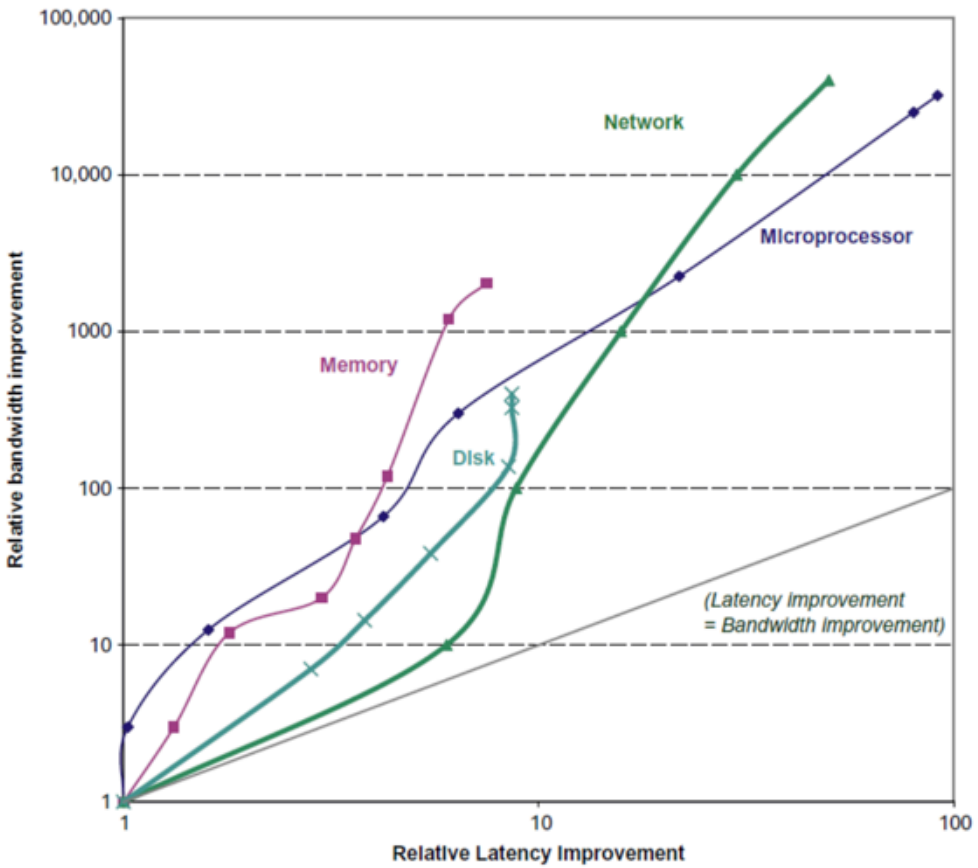
Тенденции развития технологий

- емкость магнитных дисков: в последнее время темпы роста замедлились до 5%/год
  - увеличение плотности может оказаться невозможным, возможно увеличение с 7 до 9 пластин
  - в 8-10 раз дешевле/бит по сравнению с Flash
  - в 200-300 раз дешевле/бит, чем DRAM

Пропускная способность и латентность

- Пропускная способность (Bandwidth)
  - общая работа, выполненная за определенное время
  - улучшение в 32 000-40 000 раз для процессоров
  - улучшение в 300-1200 раз для памяти и дисков
- латентность или время отклика (Latency)
  - время между началом и завершением события
  - улучшение в 50-90 раз для процессоров
  - улучшение в 6-8 раз для памяти и дисков

Пропускная способность и латентность



Логарифмическая диаграмма показателей пропускной способности и латентности

Транзисторы и проводники

- размер элемента (топологический размер, feature size)
  - минимальный размер транзистора или провода в измерениях x или y
  - 10 микрон в 1971 году до 0.002 микрон в 2024 году
  - производительность транзисторов изменяется линейно
    - задержка сигнала в проводнике = f(сопротивление, емкость)
  - плотность интеграции изменяется квадратично

Транзисторы и проводники

Number of Semiconductor Manufacturers with a Cutting Edge Logic Fab										
SilTerra X-FAB Dongbu HiTek ADI Atmel Rohm Sanyo Mitsubishi ON Hitachi Cypress SkyWater Sony Infineon Sharp Freescale Renesas (NEC) Toshiba Fujitsu TI Panasonic STMicroelectronics HLMC IBM UMC SMIC AMD Samsung TSMC Intel	ADI Atmel Rohm Sanyo Mitsubishi ON Hitachi Cypress SkyWater Sony Infineon Sharp Freescale Renesas Toshiba Fujitsu TI Panasonic STM HLMC IBM UMC SMIC AMD Samsung TSMC Intel	Cypress SkyWater Sony Infineon Sharp Freescale Renesas Toshiba Fujitsu TI Panasonic STM IBM UMC SMIC AMD Samsung TSMC Intel	Renesas Toshiba Fujitsu TI Panasonic STM HLMC IBM UMC SMIC GlobalFoundries Samsung TSMC Intel	Renesas Toshiba Fujitsu TI Panasonic STM HLMC IBM UMC SMIC GF Samsung TSMC Intel	Panasonic STM HLMC IBM UMC SMIC GF Samsung TSMC Intel	IBM GF Samsung TSMC Intel	UMC SMIC GF Samsung TSMC Intel	Samsung TSMC Intel	Samsung TSMC Intel	Samsung TSMC Intel
180 nm	130 nm	90 nm	65 nm	45 nm/40 nm	32 nm/28 nm	22 nm/20 nm	16 nm/14 nm	10 nm	7 nm	5 nm

Потребляемая мощность и энергия

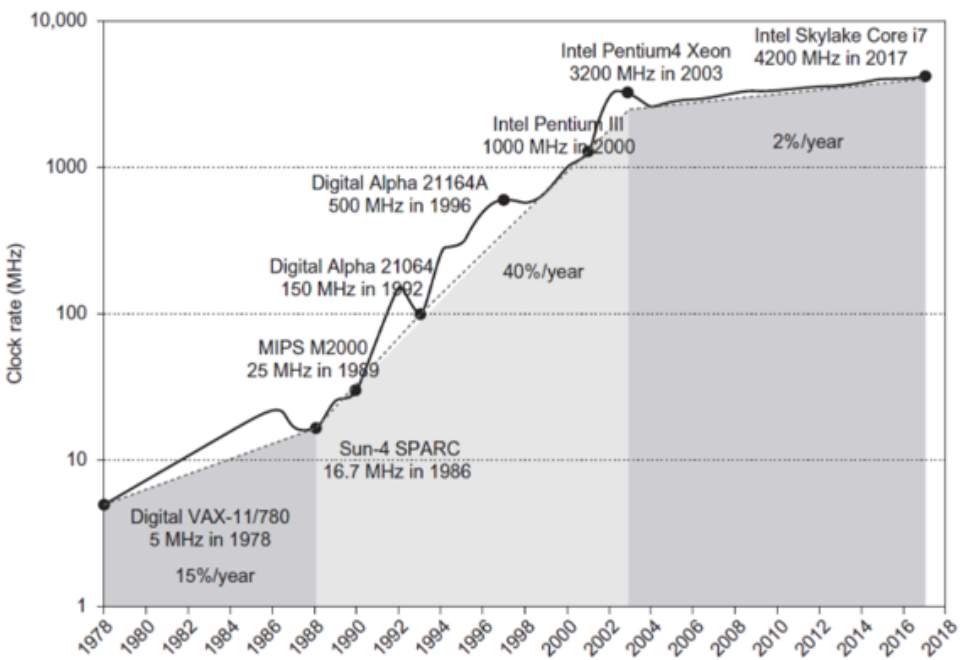
- проблема: подача и отключение электроэнергии
- Thermal Design Power (TDP)
  - характеризует устойчивое энергопотребление
  - используется в качестве цели для блока питания и системы охлаждения
  - ниже пиковой мощности (в 1,5 раза выше), выше среднего энергопотребления
- тактовая частота может быть динамически снижена для ограничения энергопотребления
- энергия на задачу часто является более точным показателем

Динамическая энергия и потребляемая мощность

- динамическая энергия
  - переключение транзистора с 0 на 1 или 1 на 0
  - $\frac{1}{2} \times C_{\text{load}} \times V^2$
- динамическая мощность
  - $\frac{1}{2} \times C_{\text{load}} \times V^2 \times f_{\text{switch}}$
- с тактовой частотой уменьшается мощность, но не энергия

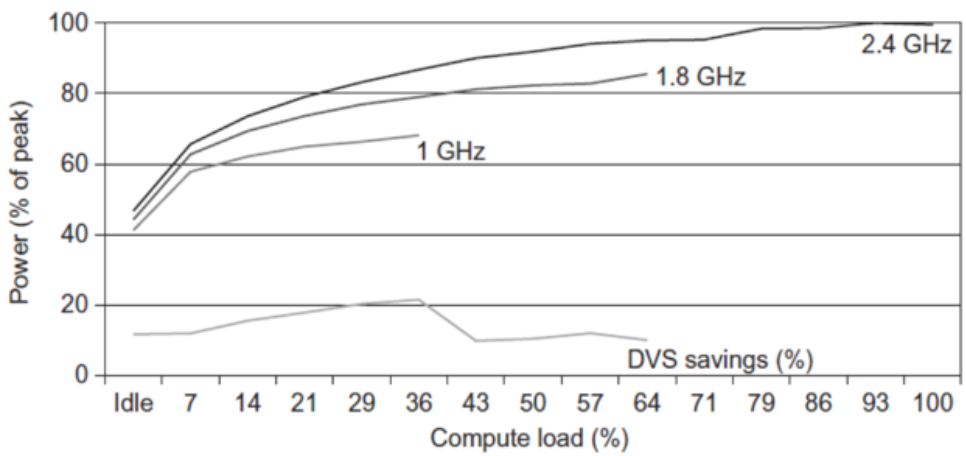
Потребляемая мощность

- Intel 80386 потреблял около 2 Вт
- Intel Core i7 3,3 ГГц потребляет 130 Вт
- тепло должно отводиться от 1,5 × 1,5 см чипа
- это предел того, что может быть охлаждено воздухом



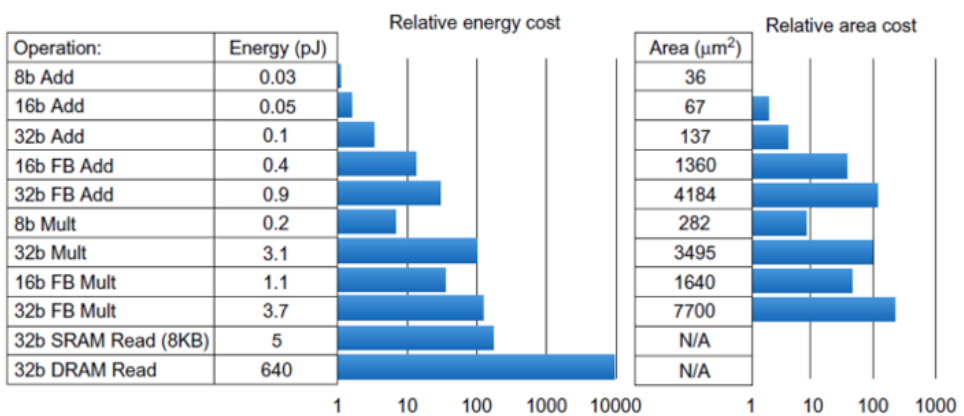
Снижение потребляемой мощности

- методы снижения мощности:
  - Do nothing well
  - Dynamic Voltage-Frequency Scaling
  - Low power state for DRAM, disks
  - Overclocking, turning off cores



Статическая мощность

- Статическая потребляемая мощность
  - 25-50% от общей мощности
  - $I_{\text{static}} \times V$
  - увеличивается с ростом числа транзисторов
  - для уменьшения: выключение питания (power gating)



Тенденции изменения стоимости

- стоимость снижается благодаря кривой освоения производства (learning curve)
  - выход годной продукции (yield)
- DRAM: цена тесно связана с себестоимостью
- микропроцессоры: цена зависит от объемов
  - на 10% меньше при каждом удвоении объема

Стоимость интегральной микросхемы

- Интегральная схема:

$$C_{\text{IC}} = \frac{C_{\text{die}} + C_{\text{test}} + C_{\text{pack}}}{Y_{\text{final}}}$$

$$C_{\text{die}} = \frac{C_{\text{wafer}}}{N_{\text{die}} \times Y_{\text{die}}}$$

$$N_{\text{die}} = \frac{\pi \times \left(\frac{D_{\text{wafer}}}{2}\right)^2}{A_{\text{die}}} - \frac{\pi \times D_{\text{wafer}}}{\sqrt{2 \times A_{\text{die}}}}$$

- Формула Бозе-Эйнштейна (эмпирическая) [Sydow, 2006]:

$$Y_{\text{die}} = Y_{\text{wafer}} \times \frac{1}{(1 + D_{\text{defects}} \times A_{\text{die}})^N}$$

- $D_{\text{defects}} = 0.016 - 0.057$ дефектов на квадратный см (2010)
- $N =$ фактор сложности процесса $= 11.5 - 15.5$ (40 nm, 2010)

Системная надежность

- надежность модуля
  - среднее время наработки на отказ (MTTF, Mean Time To Failure)
  - среднее время ремонта (MTTR, Mean Time To Repair)
  - среднее время наработки на отказ (MTBF, Mean Time Between Failures) = MTTF + MTTR
  - доступность = MTTF / MTBF

Измерение производительности

- типичные показатели производительности:
  - время отклика
  - пропускная способность
- ускорение X по сравнению с Y
  - $\frac{T_Y}{T_X}$ где T — время выполнения
- время выполнения
  - время настенных часов: включает все накладные расходы системы
  - время процессора: только время вычислений
- бенчмарки
  - ядра (например, умножение матриц)
  - программы-игрушки (например, сортировка)
  - синтетические эталоны (например, Dhrystone)
  - наборы бенчмарков (например, SPEC06fp, TPC-C)

Принципы проектирования компьютеров

- используйте преимущества параллелизма
  - например, несколько процессоров, дисков, банков памяти, конвейеризация, несколько функциональных блоков
- принцип локальности
  - повторное использование данных и инструкций
  - выделяется 2 варианта: временная и пространственная

- предпочтение общего случая (common case)
  - закон Амдала

$$T_{\text{new}} = T_{\text{old}} \times \left((1 - F_{\text{enh}}) + \frac{F_{\text{enh}}}{S_{\text{enh}}} \right)$$

$$S_{\text{total}} = \frac{1}{(1-F_{\text{enh}}) + \frac{F_{\text{enh}}}{S_{\text{enh}}}}$$

где T — время выполнения, F — доля улучшенной части, S — ускорение улучшенной части

Принципы проектирования компьютеров

- уравнение производительности процессора

$$T_{\text{CPU}} = N_{\text{cycles}} \times T_{\text{cycle}}$$

$$T_{\text{CPU}} = \frac{N_{\text{cycles}}}{f_{\text{clock}}}$$

$$CPI = \frac{N_{\text{cycles}}}{N_{\text{inst}}}$$

$$T_{\text{CPU}} = N_{\text{inst}} \times CPI \times T_{\text{cycle}}$$

$$\frac{N_{\text{inst}}}{P} \times \frac{N_{\text{cycles}}}{N_{\text{inst}}} \times T_{\text{cycle}} = T_{\text{CPU}}$$

Принципы проектирования компьютеров

- различные типы инструкций имеют разные CPI

$$N_{\text{cycles}} = \sum_{i=1}^{n} IC_i \times CPI_i$$

$$T_{\text{CPU}} = N_{\text{cycles}} \times T_{\text{cycle}}$$

ISA. ТИПЫ ДАННЫХ

Типы данных

- целочисленные
  - бинарные целочисленные
  - двоично-десятичные
  - бинарные вещественные
    - IEEE 754-1985/1997/2008/ **2019**
    - Cray
    - Intel Extended Precision (80-бит)
  - десятичные вещественные
- упакованные векторные
- адресные

Целочисленный тип данных

- разрядность: 8, 16, 32, 64 бита
- знаковые и беззнаковые
  - знаковые данные представляются в дополнительном коде
- пример знаковых чисел:

Число	**Разрядность**	**Код**
−2	8	1111 1110
−2	16	1111 1111 1111 1110

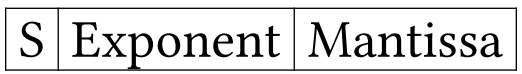
Двоично-десятичный тип данных

- разрядность: 8, 16, 32, 64 бита
- каждая четверка бит представляет десятичную цифру
- пример:

Число	Разрядность	Код
5	8	0000 0101
1234	16	0001 0010 0011 0100

Бинарный вещественный тип данных (IEEE 754-2019)

- формат:



- разрядность:

Тип	**Разрядность**	**Сегн**	**Экспонента**
Half float	16	10	5
Float	32	23	8
Double	64	52	11
Long double	**80**	**63**	**15**
Quad float	128	112	15
Десятичная точность	256	236	19

Десятичный вещественный тип данных

- формат бинарного вещественного типа:

$$(-1)^{\text{sign}} \times \text{mantissa} \times 2^{\text{exponent}}$$

- формат десятичного вещественного типа:

$$(-1)^{\text{sign}} \times \text{mantissa} \times 10^{\text{exponent}}$$

- причина:
  - повышение точности биназных вычислений

Адресный тип данных

- регистры MDR и MAR
- разрядность: 16-bit, 24-bit, 32-bit, 48-bit, 64-bit

Типы данных (векторные)

- вектор — это набор однотипных элементов в едином блоке.

Element 1	Element 2	…	Element N

- «размер вектора» = «размер элемента» × «количество элементов»

Типы данных (векторные)

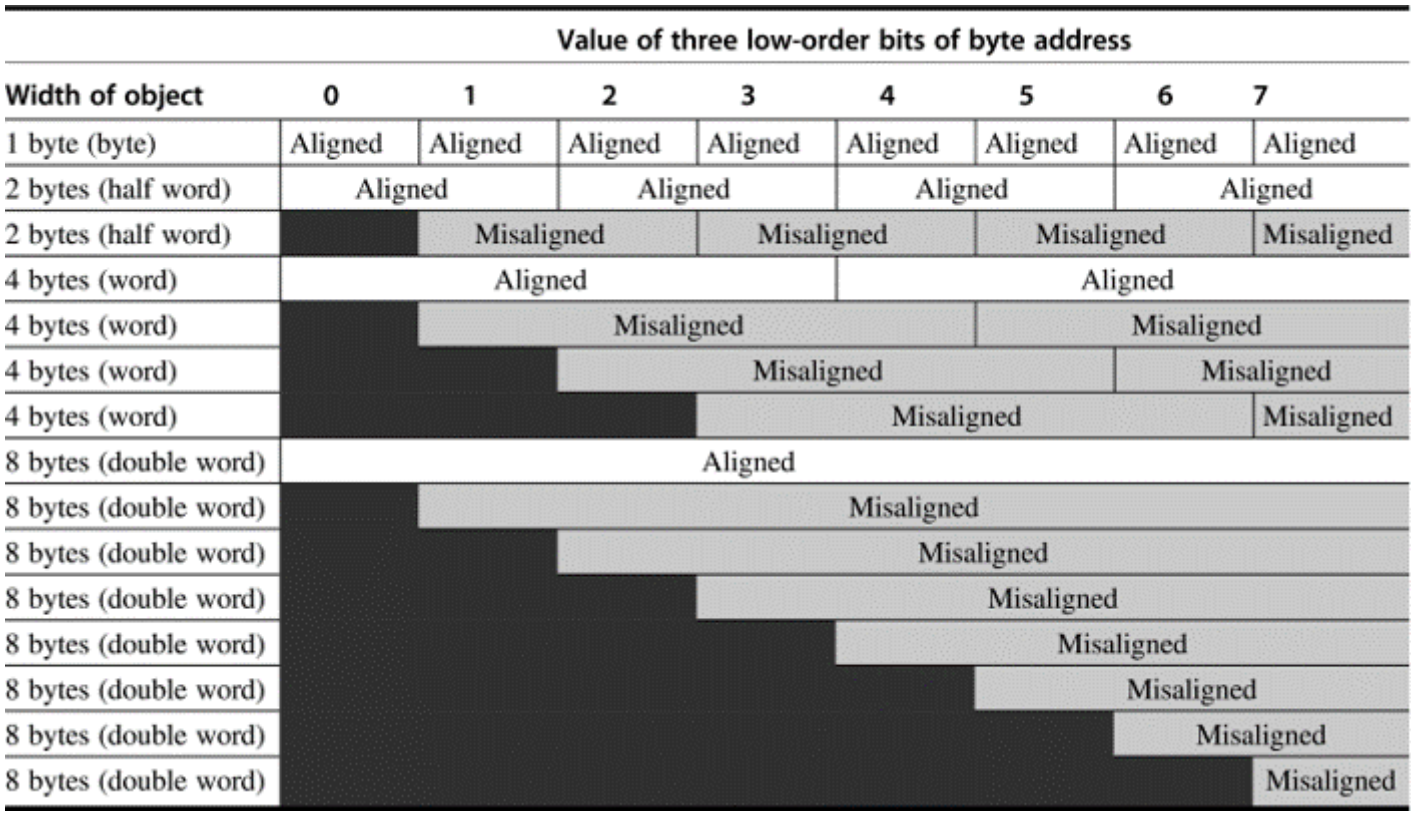
- размеры вектора: 64, 128, 256 бит.
- тип элемента вектора: целочисленный и вещественный
- разрядность элемента вектора: 8, 16, 32, 64 бита

Формат адреса операнда в памяти

- адрес операнда в памяти выровнен, если выполняются следующие условия:
  - $a \bmod s = 0$ где a — адрес операнда, s — размер операнда
- пример:

Тип	**Размер**	**Адрес**
int	4 байта	...00
double	8 байт	...000

Формат данных



	Value of three low-order bits of byte address							
Width of object	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
1 byte (byte)	Aligned	Aligned	Aligned	Aligned	Aligned	Aligned	Aligned	Aligned
2 bytes (half word)	Aligned		Aligned		Aligned		Aligned	
2 bytes (half word)		Misaligned		Misaligned		Misaligned		Misaligned
4 bytes (word)	Aligned				Aligned			
4 bytes (word)		Misaligned				Misaligned		
4 bytes (word)			Misaligned				Misaligned	
4 bytes (word)				Misaligned				Misaligned
8 bytes (double word)	Aligned							
8 bytes (double word)		Misaligned						
8 bytes (double word)			Misaligned					
8 bytes (double word)				Misaligned				
8 bytes (double word)					Misaligned			
8 bytes (double word)						Misaligned		
8 bytes (double word)							Misaligned	
8 bytes (double word)								Misaligned

Порядок байт в слове

- от младшего к старшему (*little-endian*)
- от старшего к младшему (*big-endian*)
  - пример расположения int-а в памяти
  - байт 0 младший, байт 3 старший

Особенности *little — endian*

- в памяти расположено int число $0x00000003$ по адресу

Адрес	**Значение памяти**	**16 битное число из**	**8 битное число из**
s+3	00		
s+2	00		
s+1	00	00	
s	03	03	03

ISA. СПОСОБЫ АДРЕСАЦИИ ОПЕРАНДОВ

Где находятся операнды инструкции?

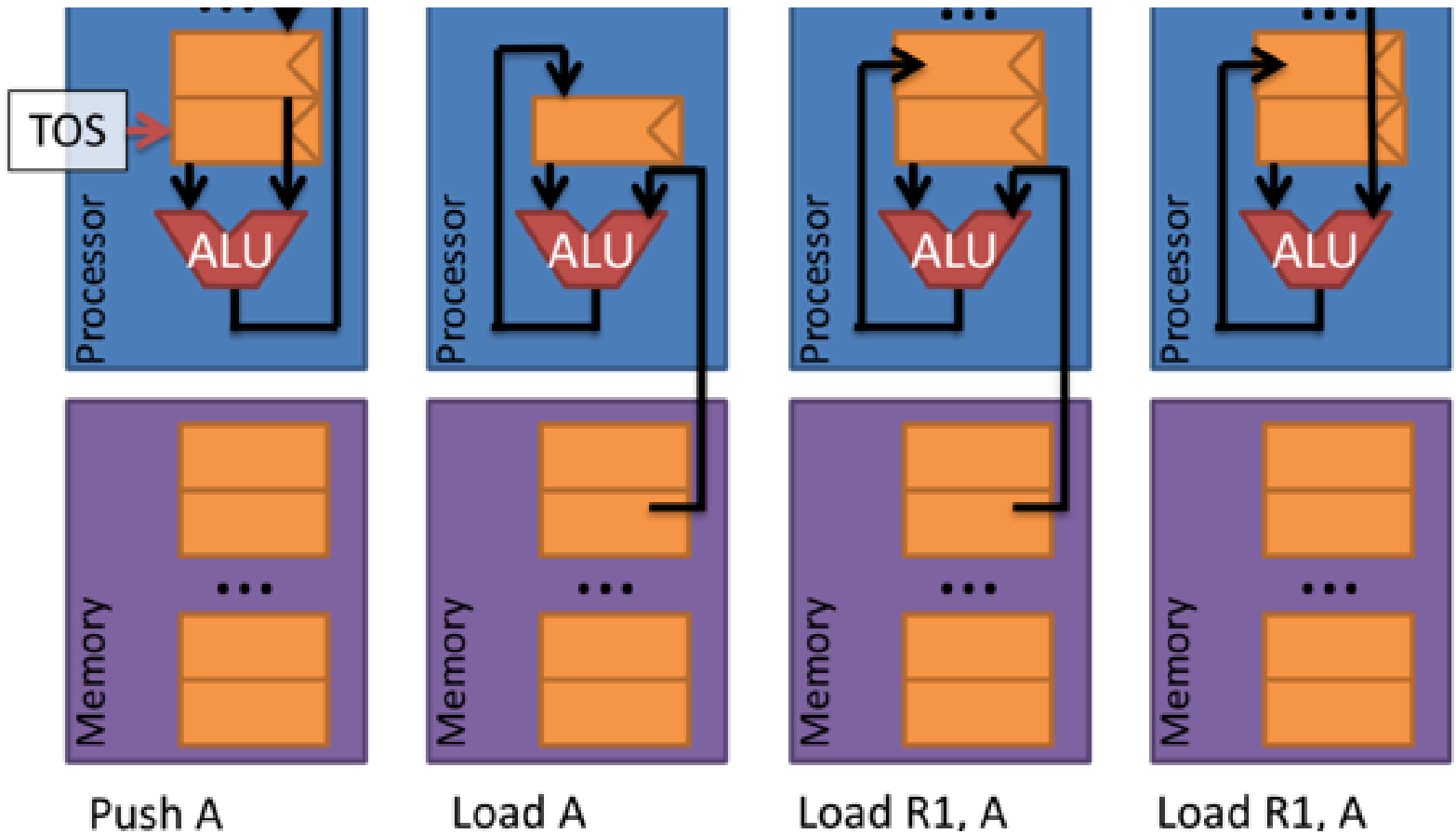
- 2 — регистры
- 2 — память

Ссылки на типы ISA

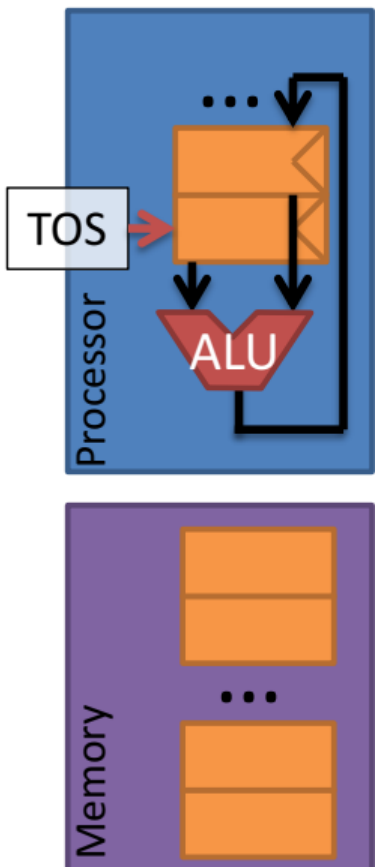
Number of memory addresses	Maximum number of operands allowed	Type of architecture	Examples
0	3	Load-store	ARM, MIPS, PowerPC, SPARC, RISC-V
1	2	Register-memory	IBM 360/370, Intel 80x86, Motorola 68000, TI TMS320C54x
2	2	Memory-memory	VAX (also has three-operand formats)
3	3	Memory-memory	VAX (also has two-operand formats)

Типы адресации

C = A + B



Примеры режимов ISA



- HP 3000
- Intel x87 Floating Point Unit
- Java Virtual Machine

Типы адресации

- Непосредственная (Immediate)
  - операнд в теле инструкции
- Регистровая (Register)
  - в инструкции хранится адрес регистра, который содержит операнд
- Пример:

Тип адресации	**Инструкция**	**Действие**
Регистровая	add r1, r2, r3	$\text{Reg}\,[r1] = \text{Reg}\,[r2] + \text{Reg}\,[r3]$
Непосредственная	add r1, r2, 3	$\text{Reg}\,[r1) = \text{Reg}\,[r2] + 3$

Типы адресации операндов в памяти

Название режима адресации	**Формула адреса**	**Пример инструкции**	**Выполняемое действие**
Непосредственная (Displacement)	Reg [Base] + Disp.	add r1, r2, 100(r3)	Reg $[r1]$ = Reg $[r2]$ + Mem $[100 +$ Reg $[r3]]$
Базовая или Относительная регистровая (Register Indirect)	Reg [Base)	add r1, r2, (r3)	Reg $[r1)$ = Reg $[r2)$ + Mem [Reg $[r3]]$
Базовая индексная (Indexed)	Reg [Base] + Reg [Index]	add r1, r2, (r3+r4)	Reg $[r1]$ = Reg $[r2]$ +

			Mem [Reg [$r3$) + Reg [$r4$]]
Базовая индексная со смещением	Reg [Base] + Reg [Index] + Disp.	add r1, r2, 100(r3)[r4]	Reg [$r1$] = Reg [$r2$) + Mem [Reg [$r3$] + Reg [$r4$] + 100]
Прямая или Абсолютная (direct or absolute)		add r1, r2, (100)	Reg [$r1$] = Reg [$r2$] + Mem [100]
Относительная (Memory Indirect)	Mem [Mem [Reg [r]]]	add r1, r2, @(r3)	Reg [$r1$] = Reg [$r2$] + Mem [Mem [Reg [$r3$]]]

Типы адресации операндов в памяти (продолжение)

- Адресация с масштабированием
  - Характеризуется наличием масштабирующего коэффициента. Может
  - считаться со следующими типами адресации: индексная,
  - базово-индексная, базово-индексная со смещением, абсолютная
  - Коэффициент равен размеру базового типа (2, 4, 8).
  - Хранится в коде инструкции
  - Intel SIMD
- Пример:

Тип адресации	**Инструкция**	**Действие**
Абсолютная адресация базово-индексная	add r1, r2, [r3+4*r4]	$\text{Reg}\ [r1] = \text{Reg}\ [r2] + \text{Mem}\ [\text{Reg}\ [r3] + 4 \times \text{Reg}\ [r4]]$

Типы адресации операндов в памяти (продолжение)

- Адресация с авто модификацией
  - изменяет значение базового регистра на заданное число.
  - число задается непосредственно или хранится в регистре.
  - бывает постфиксная и префиксная.
  - применяется совместно с базовой, индексной или базово-индексной адресацией.
  - пример:

Тип адресации	**Инструкция**	**Действие**
Базовая с пост модификацией	add r1, r2, (r3)+	$\text{Reg}\ [r1] = \text{Reg}\ [r2] + \text{Mem}\ [\text{Reg}\ [r3]]; \text{Reg}\ [r3] = \text{Reg}\ [r3] + \text{Disp}.$

Типы адресации операндов в памяти (окончание)

- Относительный режим
  - индексный регистр, где в качестве базового регистра используется
  - счетчик команд.
  - Пример:

Тип адресации	**Адрес**
Относительный индексный	$\mathrm{Ea} = \mathrm{PC} + \mathrm{Disp.}$
Относительный базово-индексный	$\mathrm{Ea} = \mathrm{PC} + \mathrm{Reg}\ [r]$

ISA. ИНСТРУКЦИИ

Задачи, решаемые при проектировании набора инструкций

- функциональная полнота
- простота декодирования
- уменьшение объема программного кода
- достижение высокого быстродействия

Классы инструкций

- передача данных
  - LD, ST, MFC1, MTC1, MFC0, MTC0
- арифметико-логические
  - ADD, SUB, AND, OR, XOR, MUL, DIV, SLT, LUI
- управление потоком исполнения
  - BEQZ, JR, JAL, TRAP, ERET
- для работы с плавающей запятой
  - ADD.D, SUB.S, MUL.D, C.LT.D, CVT.S.W
- мультимедийные (SIMD)
  - ADD.PS, SUB.PS, MUL.PS, C.LT.PS
- строковые
  - REP MOVSB (x86)
- системные
  - вызовы!
  - для работы с виртуальной памятью
- для работы с графикой
  - операции для работы с пикселями и вертексами
  - операции сжатия/восстановления

- Different Classes of CPU Registers

Типы инструкций передачи данных

- инструкции загрузки и сохранения данных из/в память
- инструкции загрузки констант в регистры
- безусловные инструкции пересылки между регистрами процессора
- условные инструкции пересылки

Характеристики инструкций загрузки и сохранения данных из/в память

- разрядность данных: 8, 16, 32, 64, 128, 256 бит
- тип загружаемых данных:
  - целочисленный беззнаковый
  - целочисленный знаковый
  - вещественный
  - векторный
- тип используемой адресации
- порядок байт в слове

Отличие знаковой и беззнаковой загрузки целочисленных данных

- беззнаковая инструкция загрузки дополняет старшие биты нулями
- знаковая инструкция загрузки дополняет старшие биты знаковым битом
- пример:

Тип загрузки	**Число в памяти (8 бит)**	**Число в регистре (32 бита)**
беззнаковая	1111 1110 (0xFE)	0x00 00 00 FE
знаковая	1111 1110 (0xFE)	0xFF FF FF FE

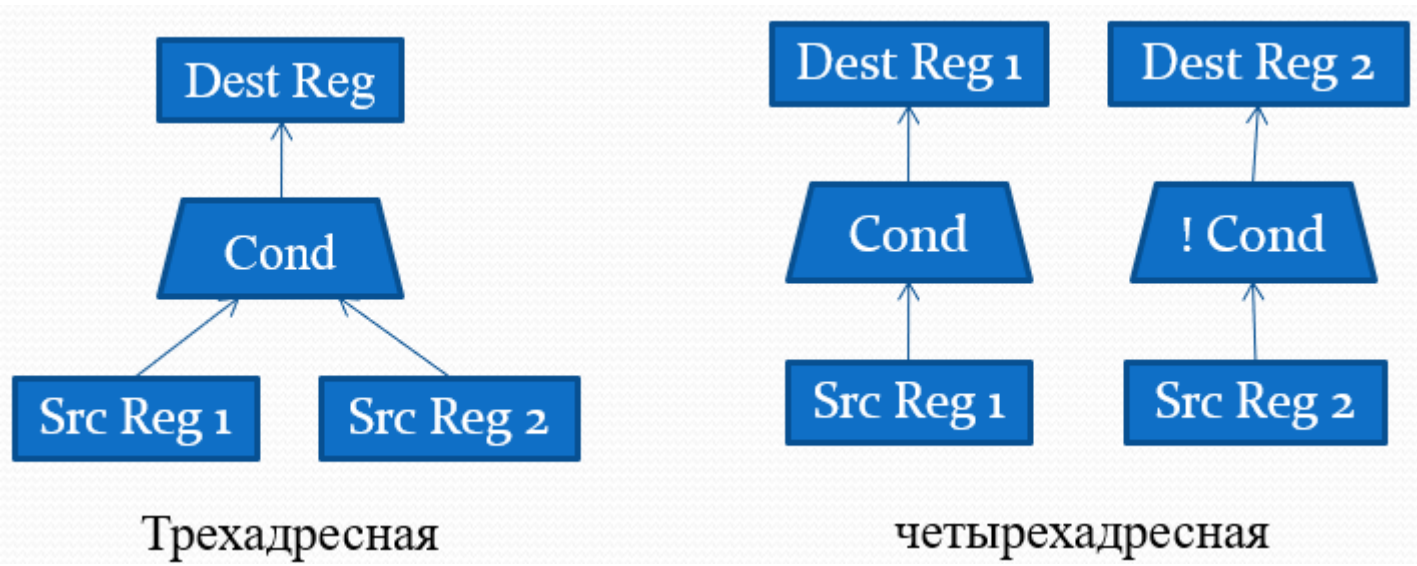
Дополнительные характеристики инструкций пересылки

- характеристики инструкций загрузки констант
  - разрядность константы
  - способ расширения константы до требуемого размера
- характеристики инструкций безусловной пересылки между регистрами процессора
  - типы регистров, между которыми возможна прямая пересылка данных

Характеристики инструкции условной пересылки между регистрами процессора

- тип: трехадресный, четырехадресный
- способ задания условия
- пример:

```
move dest reg, src reg1, src reg2
```


Dest Reg
Cond
Src Reg 1
Src Reg 2
Трехадресная
Dest Reg 1
Dest Reg 2
Cond
! Cond
Src Reg 1
Src Reg 2
четырехадресная

Использование инструкций условной пересылки

Код на C

```
If (condition)
  a = b;
Else
  a = c;
```

Код на ассемблере

```
cmp condition
je m1
mov a,c
jmp m2
m1: mov a,b
m2:
```

Код на ассемблере с инструкцией условной пересылки

```
cmp condition
move a,b,c
```

Характеристики арифметико-логических инструкций

- количество операндов
  - двухадресные
    - Operand1 = Operand1 OPERATION Operand2
  - трехадресные
    - Operand3 = Operand1 OPERATION Operand2
- разрядность операндов
  - для современных 64-х битных архитектур составляет обычно 32 и 64 бита
  - исключение x86/x86-64
- адресация операндов
- генерация исключений и модификация флагов состояния процессора

Инструкции управления потоком исполнения

- инструкции перехода
  - безусловные (jump)
  - условные (branch)
- выполняемые функции
  - вызов процедур
  - возврат из процедуры
  - организация циклов
  - условное ветвление
  - безусловный переход

Адресация точки перехода

- адрес перехода вычисляется на размер команды (исключение x86)
- режимы адресации и их характеристики:
  - абсолютная
    - EA = d + Addr
    - размер поля адреса
  - относительная
    - EA = PC + d $\times$ Disp
    - размер поля смещения
  - косвенная регистровая
    - EA = Reg
    - универсальный регистр или специализированный регистр связывания

Связывание

- это процесс сохранения адреса инструкции перехода или следующей за ней инструкции
- может использоваться с любым типом перехода и обычно с относительным режимом адресации
- куда сохраняются данные:
  - регистр связывания
    - универсальный регистр
    - неявно заданный универсальный регистр
    - специальный регистр
  - стек (память)

Реализация и недостатки подходов к связыванию

- использование стека
  - указатель стека является аппаратным. Проходит задачу программиста
  - недостатки: усложняет микроархитектуру, не подходит для суперскалярных процессоров, требует выборки из памяти
- регистр связывания
  - компилятор или программист должны самостоятельно организовать хранение адреса возврата из процедуры

Особенности задания условия для инструкций условного перехода

- через сравнение регистра с нулем
  - совмещается с командой установки
- задание условия на основе регистра флагов
  - в инструкции может явно задаваться требуемое состояние флагов в поле условия
- сравнение двух регистров на основе предиката

Примеры различных способов задания условий

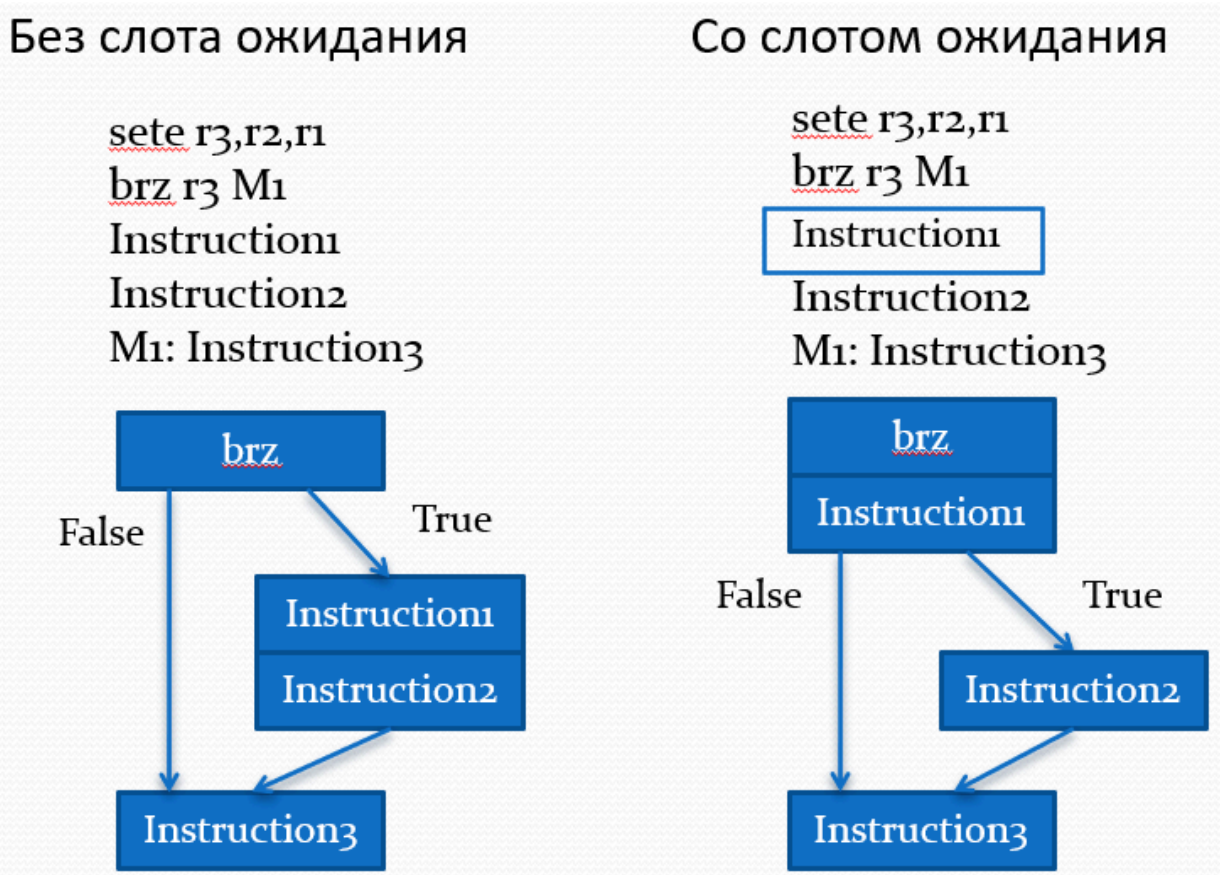
- через сравнение регистра с нулем
  - `setl r3,r2,r1`
  - `bnz r3 m1`
- задание условия на основе регистра флагов
  - `cmp eax,ebx`
  - `je m1`
- задание условия в инструкции
  - `bg r3,r2 m1`

какие недостатки у данных способов?

Слот ожидания

- это инструкция, расположенная непосредственно за инструкцией условного перехода, которая может быть выполнена независимо от условия
- переход со слотом ожидания называется отложенным
- был введён для уменьшения задержки исполнения инструкции условного перехода

Слот ожидания (пример)



Управление механизмом предсказания ветвления

- статическое
  - всегда предсказывать выполнение перехода
  - всегда предсказывать не выполнение перехода
- динамическое (влияет только в случае отсутствия истории перехода)
  - предсказывать выполнение перехода
  - предсказывать не выполнение перехода

Предиктирование

- процесс управления потоком команд, основанный на предиктировании исполнения инструкций
- это величины, которые принимают значение а или б; для их хранения используются специальные или универсальные регистры
- предиктированная инструкция используется, если связанный с ней предикат истинен. В противном случае инструкция просто пропускается

Предиктирование и унификация инструкций

- предиктирование
  - в каждой инструкции содержится условие. Если условие истинно, инструкция выполняется, иначе — нет
- нуллификация
  - в инструкции может явно задаваться требуемое состояние флагов в поле условия
- сравнение двух регистров на основе предиката

Использование предиктирования

```
setg rp1, r1, r2
add r5, r1, r3
add r6, r1, r4
rp1: add r1, r5, 2
!rp1: add r1, r6, 1
```

```
brg r1, r2 m1:
add r1, r1, r4
add r1, r1, 1
jmp m2
m1: add r1, r1, r3
add r1, r1, 2
m2:
```

Плотность кода (англ. Code density)

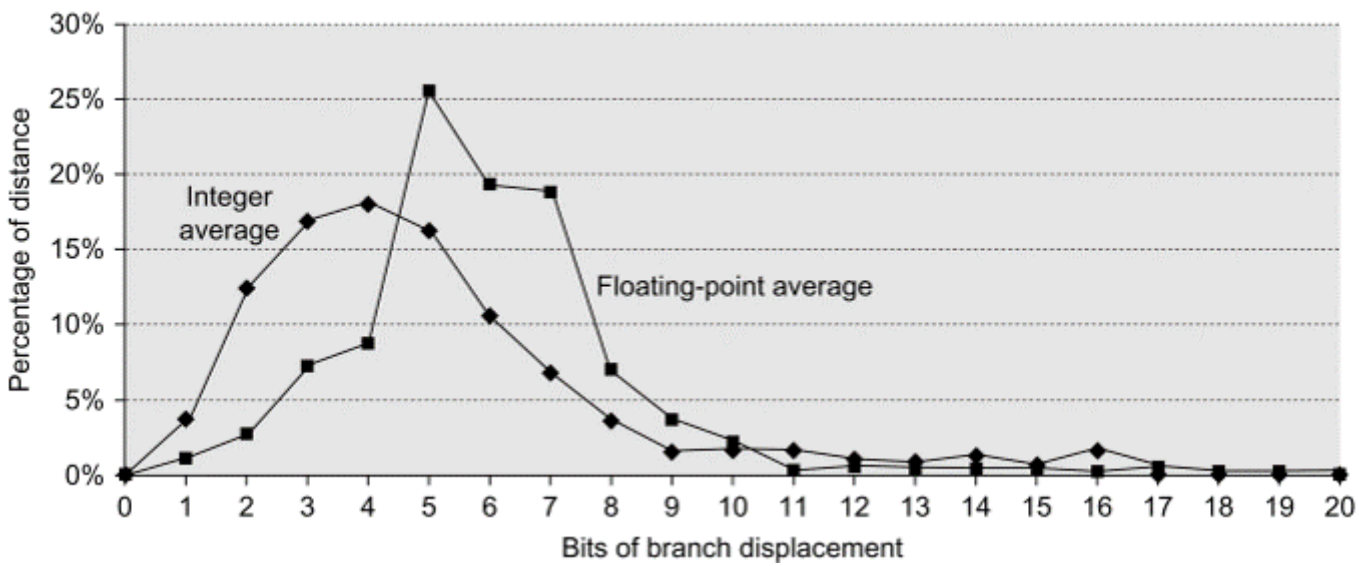
- определяет количество элементарных операций, размещённых в единице объёма памяти

Top10 инструкций x86

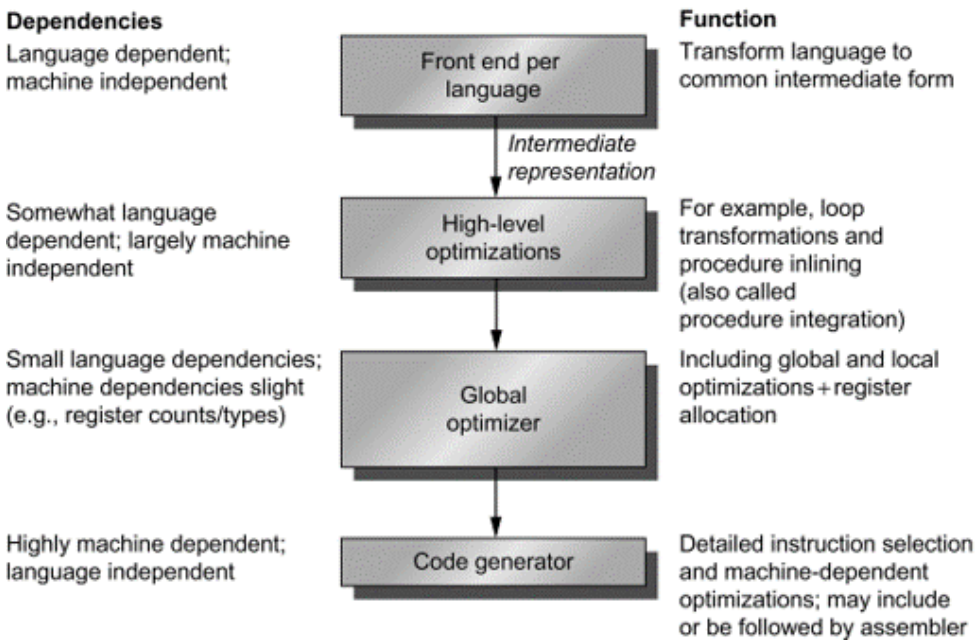
Rank	80x86 instruction	Integer average % total executed)
1	Load	22%
2	Conditional branch	20%
3	Compare	16%
4	Store	12%
5	Add	8%
6	And	6%
7	Sub	5%
8	Move register-register	4%
9	Call	1%
10	Return	1%
Total		**96%**

- простые инструкции занимают 96% от всего множества
- статистика на основе 5 программ SPECint92

Сдвиг в инструкциях между точкой перехода и инструкцией ветвления

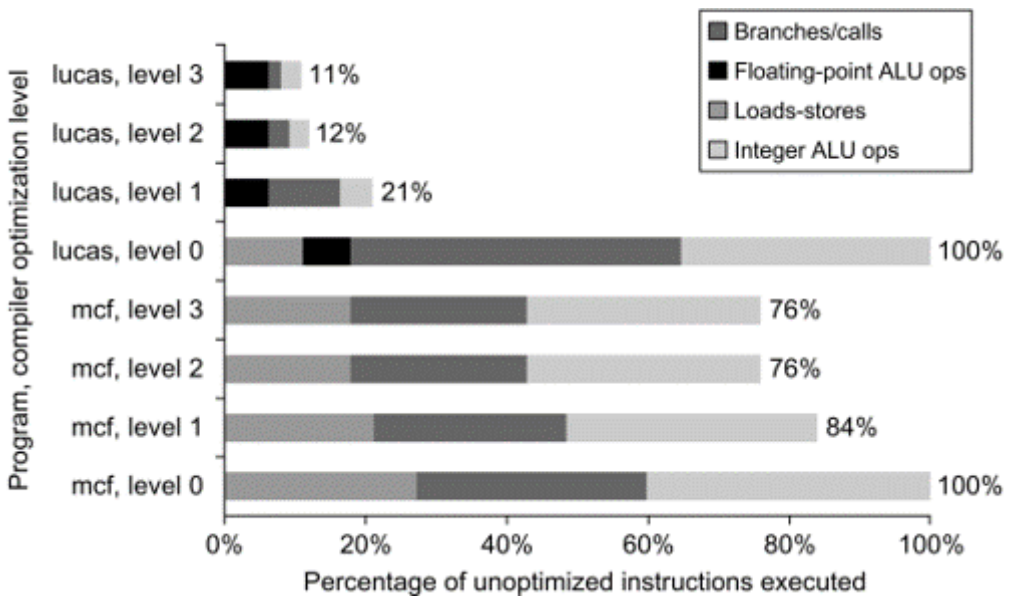


Оптимизация со стороны компилятора



Optimization name	Explanation	Percentage of the total number of optimizing transforms
High-level	*At or near the source level; processor-independent*	
Procedure integration	Replace procedure call by procedure body	N.M.
Local	*Within straight-line code*	
Common subexpression elimination	Replace two instances of the same computation by single copy	18%
Constant propagation	Replace all instances of a variable that is assigned a constant with the constant	22%
Stack height reduction	Rearrange expression tree to minimize resources needed for expression evaluation	N.M.
Global	*Across a branch*	
Global common subexpression elimination	Same as local, but this version crosses branches	13%
Copy propagation	Replace all instances of a variable A that has been assigned X (i.e., $A = X$) with X	11%
Code motion	Remove code from a loop that computes same value each iteration of the loop	16%
Induction variable elimination	Simplify/eliminate array addressing calculations within loops	2%
Processor-dependent	*Depends on processor knowledge*	
Strength reduction	Many examples, such as replace multiply by a constant with adds and shifts	N.M.
Pipeline scheduling	Reorder instructions to improve pipeline performance	N.M.
Branch offset optimization	Choose the shortest branch displacement that reaches target	N.M.

Влияние оптимизации на количество инструкций

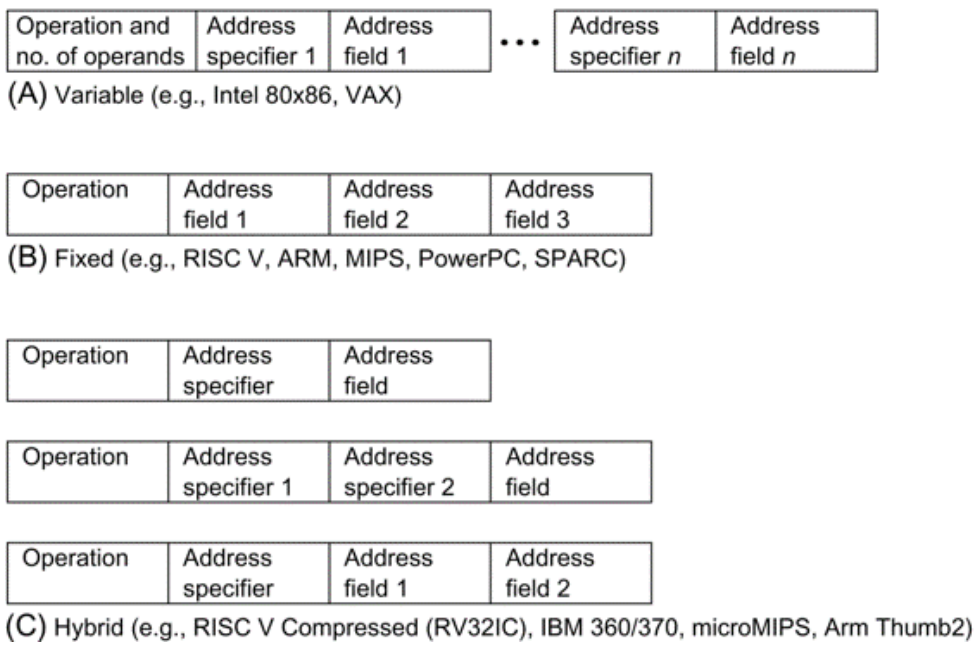


SPEC2000: программы lucas и mcf

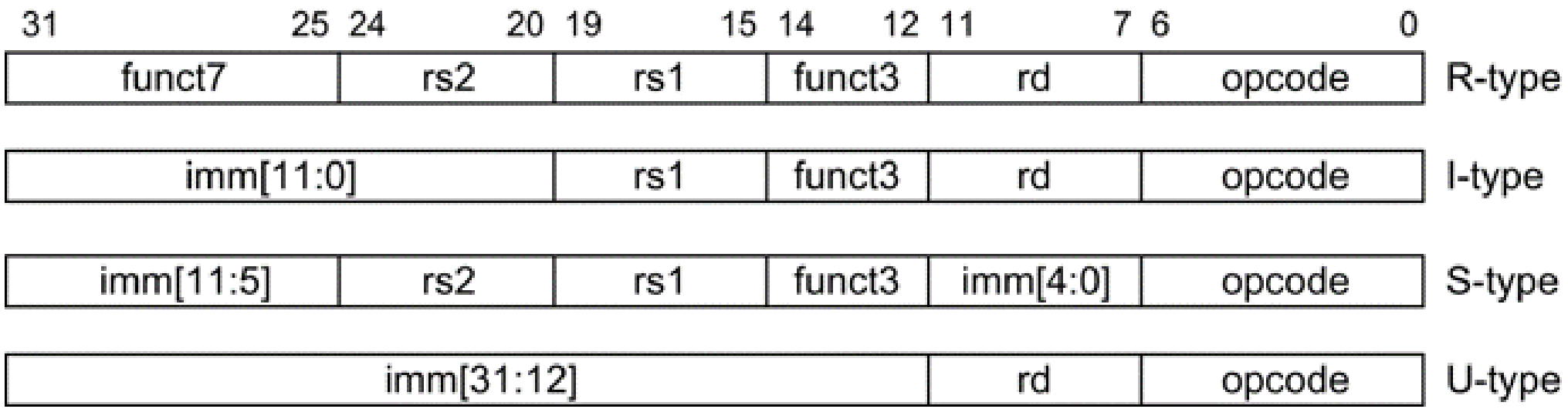
КОДИРОВАНИЕ ИНСТРУКЦИЙ

Способы кодирования инструкций

- постоянной длины
  - простота декодирования
  - RISC: MIPS (4 байта), PowerPC, SPARC, ARM…
- переменной длины
  - занимают меньше памяти
  - CISC: IBM 360, x86 (от 1 до 17 байт), Motorola 68k, VAX…
- комбинированный
  - MIPS16, THUMB (2 формата: 2 и 4 байта)



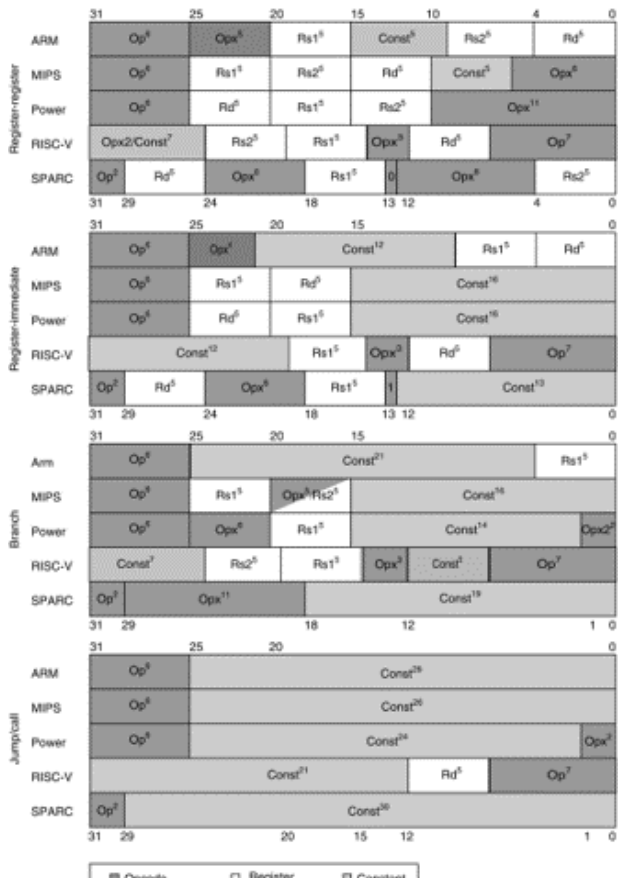
RISC-V инструкции



RISC-V инструкции

Instruction format	Primary use	rd	rs1	rs2	Immediate
R-type	Register-register ALU instructions	Destination	First source	Second source	
I-type	ALU immediates Load	Destination	First source base register		Value displacement
S-type	Store Compare and branch		Base register first source	Data source to store second source	Displacement offset
U-type	Jump and link Jump and link register	Register destination for return PC	Target address for jump and link register		Target address for jump and link

RISC-V инструкции



- Op = the main opcode
- Opx = an opcode extension
- Rd = the destination register
- Rs1 = source register 1
- s2 = source register 2
- Const = a constant (used as an immediate, address, mask, or shift amount)

Сравнение e и архитектуры для серверов и PMD

Сравнение длины инструкции

	ARMv8	MIPS64 R6	Power v3.0	RV64G	SPARCv9
Original date (base ISA)	1986	1986	1990	2016	1987
Date of this ISA	2011	2014	2013	2016	2008
Instruction size (bits)	32	32	32	32	32
Address space (size, model)	64 bits (flat)	64 bits, flat	64 bits, flat	64 bits, flat	64 bits, flat
Data alignment	Aligned preferred	Aligned preferred	Unaligned	Aligned preferred	Aligned
Data addressing modes	8 (including scaled, pre/post increment)	1 (+1 for FP only)	4	1	2
Integer registers (number, model, size)	31 GPR x 64, plus stack pointer	31 GPR × 64 bits	31 GPR × 64 bits	31 GPR × 64 bits	31 GPR × 64 bits
Separate floating-point registers	32x32 or 32x64 bits	32 × 32 or 32 × 64 bits	32 × 32 or 32 × 64 bits	32 × 32 or 32 × 64 bits	32 × 32 or 32 × 64 bits
Floating-point format	IEEE 754 single, double	IEEE 754 single, double	IEEE 754 single, double	IEEE 754 single, double	IEEE 754 single, double

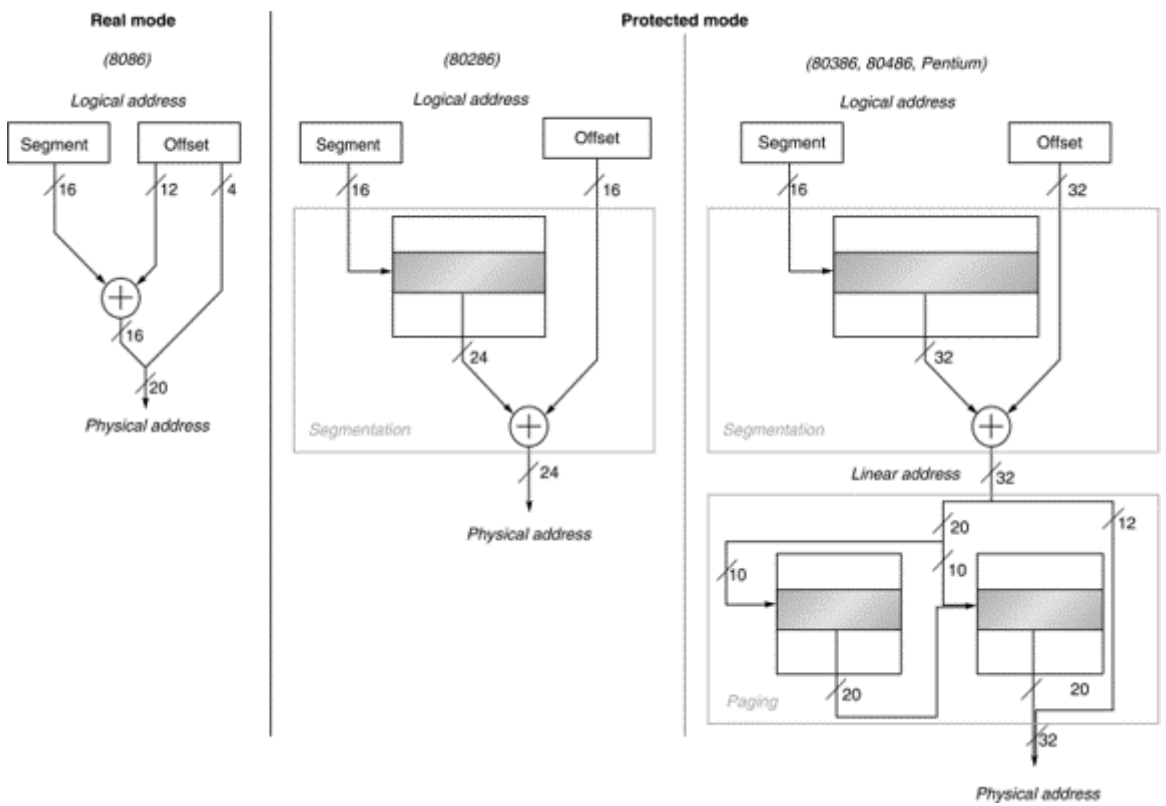
Сравнение архитектур для встраиваемых решений

	microMIPS64	RV64GC	Thumb-2
Date announced	2009	2016	2003
Instruction size (bits)	16/32	16/32	16/32
Address space (size, model)	32/64 bits, flat	32/64 bits flat	32 bits, flat
Data alignment	Aligned	Aligned, preferred	Aligned
Data addressing modes	2	1	6
Integer registers (number, model, size)	31 GPR × 64 bits	31 GPR × 64 bits	15 GPR × 32 bits
Integer registers accessible by most 16-bit instructions (which use should specifiers)	8 GPR + SP + GP +RA GPRs: 0, 2-7, 17, or 2-7, 16, 17	8 GPRs + SP GPRs: 8-15	8 GPR + SP × 32 bits

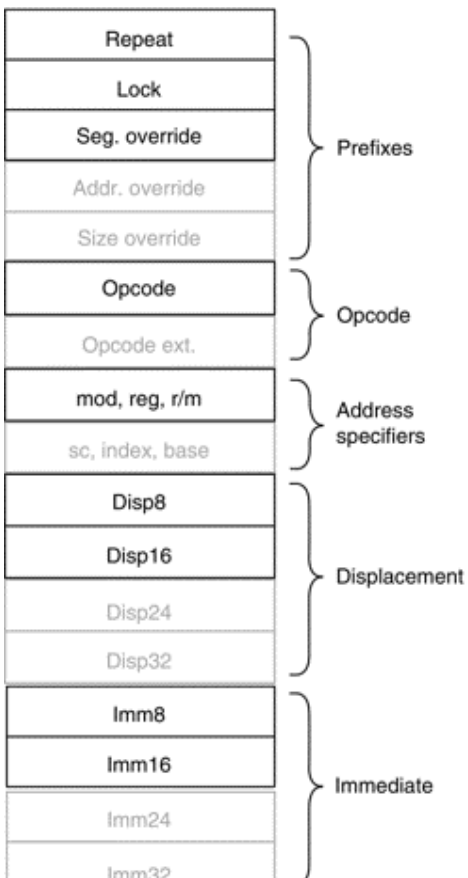
Сравнение архитектур с переменной длиной инструкции

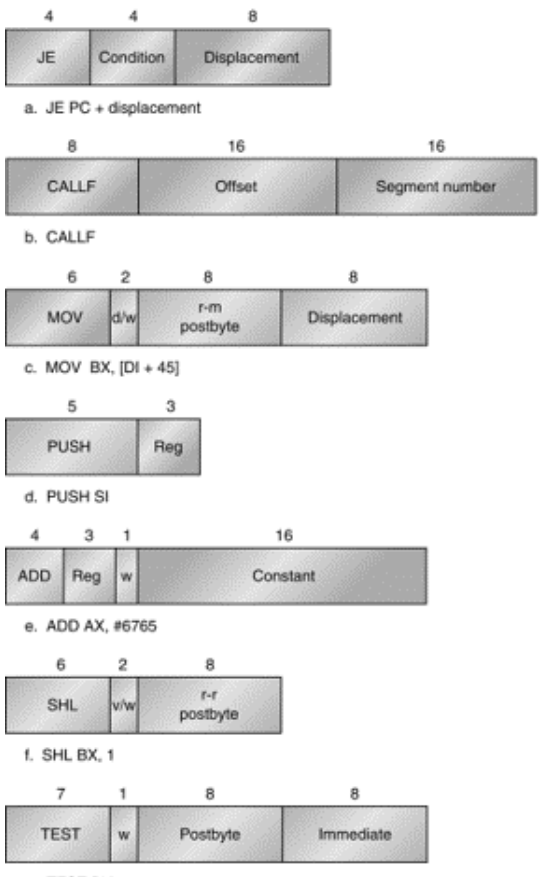
	IBM 360/370	Intel 8086	Motorola 68000	DEC VAX
Date announced	1964/1970	1978	1980	1977
Instruction size(s) (bits)	16, 32, 48	8, 16, 24, 32, 40, 48	16, 32, 48, 64, 80	8, 16, 24, 32, ... , 432
Addressing (size, model)	24 bits, flat/ 31 bits, flat	4 + 16 bits, segmented	24 bits, flat	32 bits, flat
Data aligned?	Yes 360/No 370	No	16-bit aligned	No
Data addressing modes	2/3	5	9	=14
Protection	Page	None	Optional	Page
Page size	2 KB & 4 KB	—	0.25 to 32 KB	0.5 KB
I/O	Opcode	Opcode	Memory mapped	Memory mapped
Integer registers (size, model, number)	16 GPR × 32 bits	8 dedicated data × 16 bits	8 data and 8 address × 32 bits	15 GPR × 32 bits
Separate floating-point registers	4 × 64 bits	Optional: 8 × 80 bits	Optional: 8 × 80 bits	0
Floating-point format	IBM (floating hexadecimal)	IEEE 754 single, double, extended	IEEE 754 single, double, extended	DEC

Адресация в x86

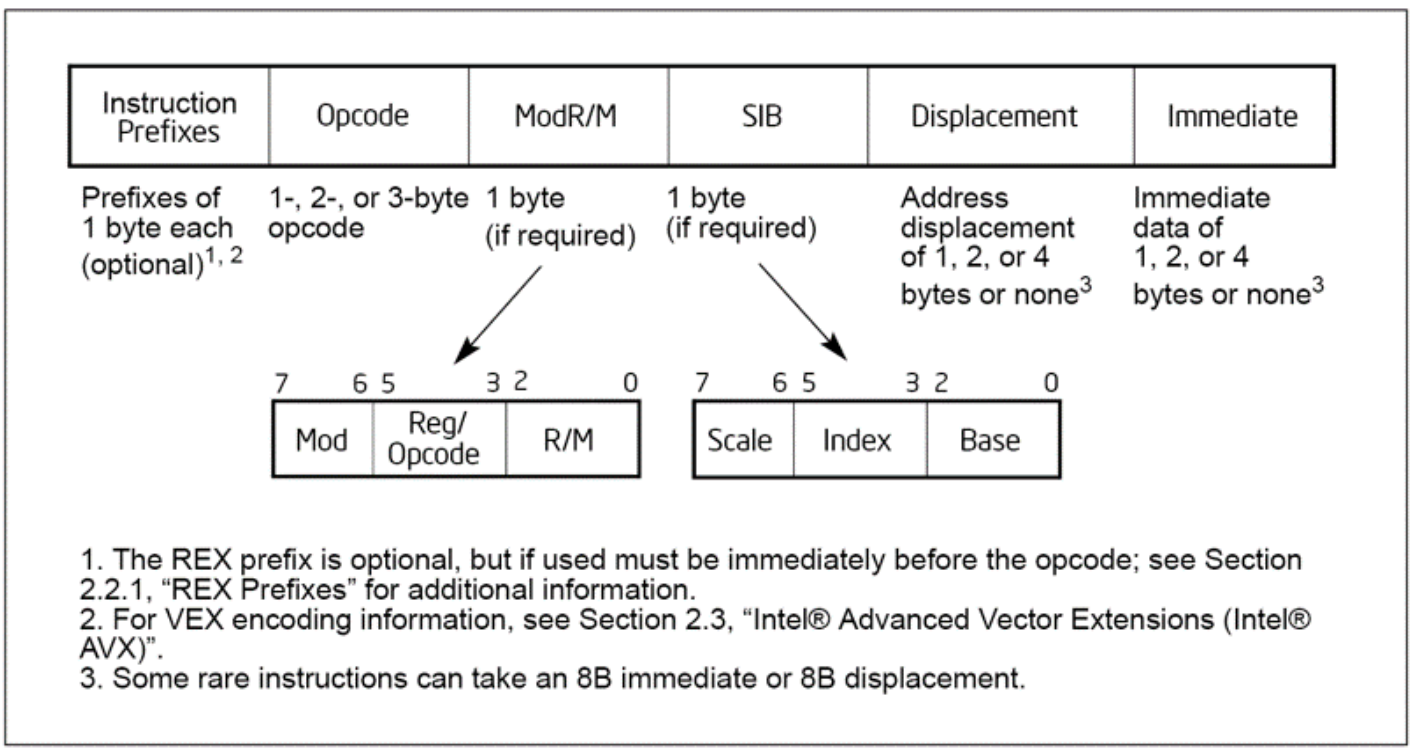


Форматы инструкций для x86 и x86-64





Кодирование инструкций x86



в зависимости от типа инструкции длина меняется от 2 до 17 байт

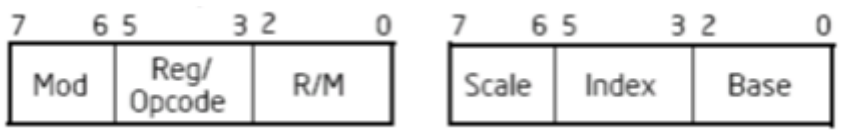
Префиксы

- это необязательная часть инструкции, которая находится перед полем кода операции
- основные типы префиксов для x86-64:
  - legacy
  - REX
  - VEX
  - EVEX

Legacy префиксы

- группа 1 — префиксы блокировки и повторения
  - содержит количество повторений для строковой инструкции, задаваемой различными командами REP
  - блокировка устанавливается при доступе к общей памяти в мультипроцессорной системе
- группа 2 — префиксы замены сегмента и оптимизации переходов
- группа 3 — префикс изменения размера операндов
  - 16-ми и 32-битные операции
- группа 4 — префикс изменения размера адреса операнда

Адресация операндов в x86

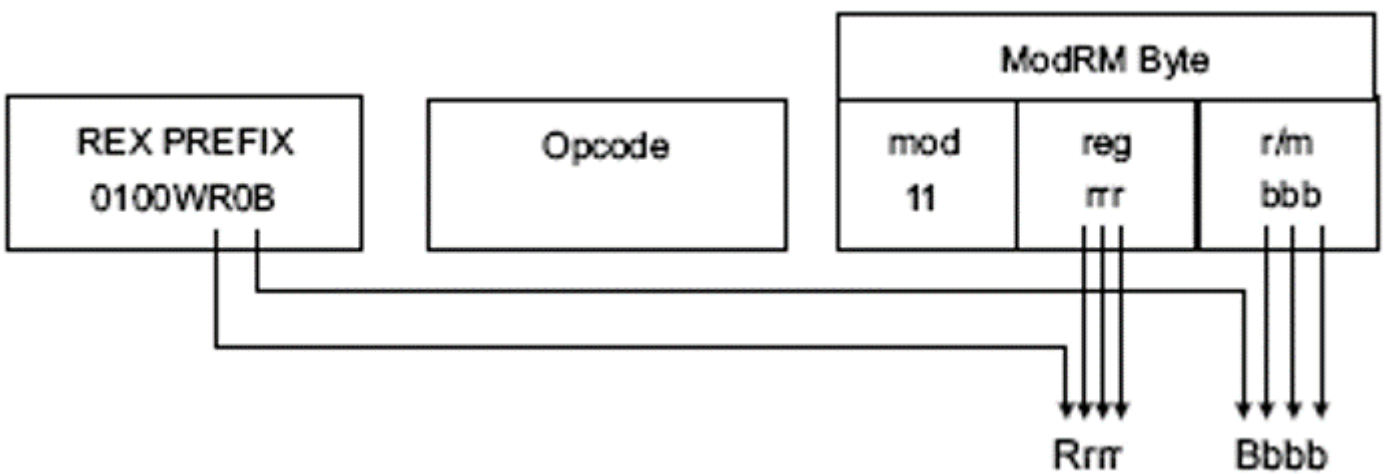


- на операции:
  - первый находится в регистре, его адрес задаётся в поле Reg/Opcode
  - адрес второго операнда задаётся полями mod, R/M, Scale, Index, Base
    - EA = Disp + Base + Index × Scale

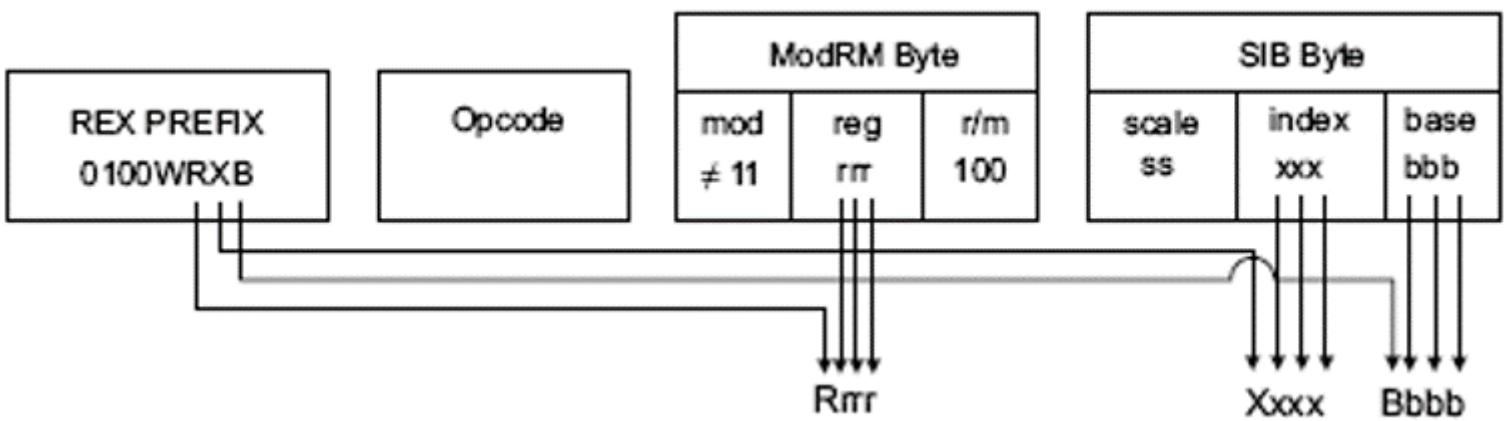
Отличия x86 и x86-64

- увеличение количества всех типов регистров в два раза (кроме x87)
- увеличение разрядности адреса до 64 бит

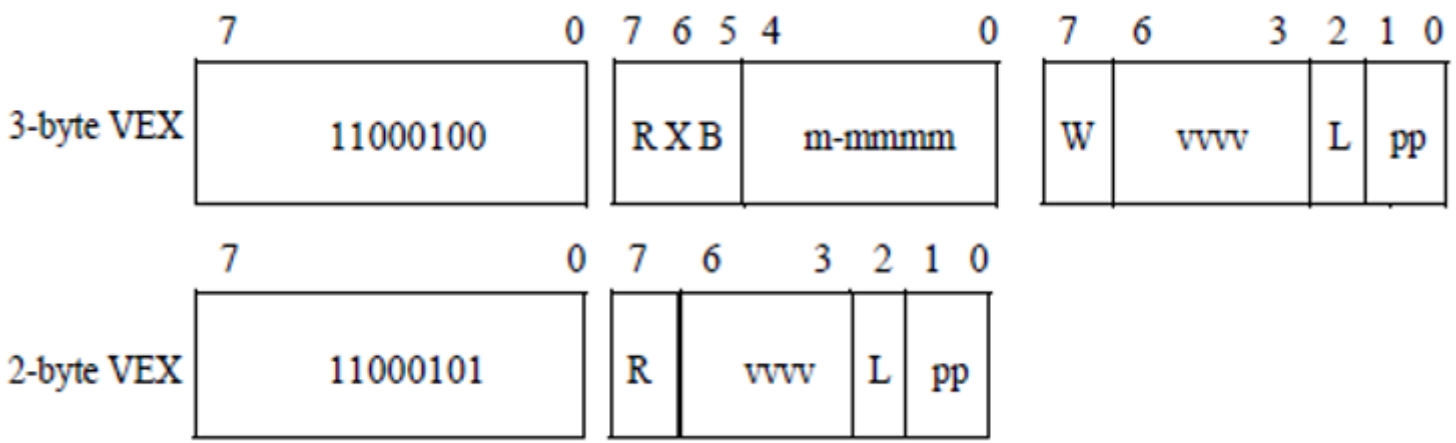
REX префикс, адресация в режиме x86-64



REX префикс, адресация в режиме x86-64 (окончание)



VEX префикс

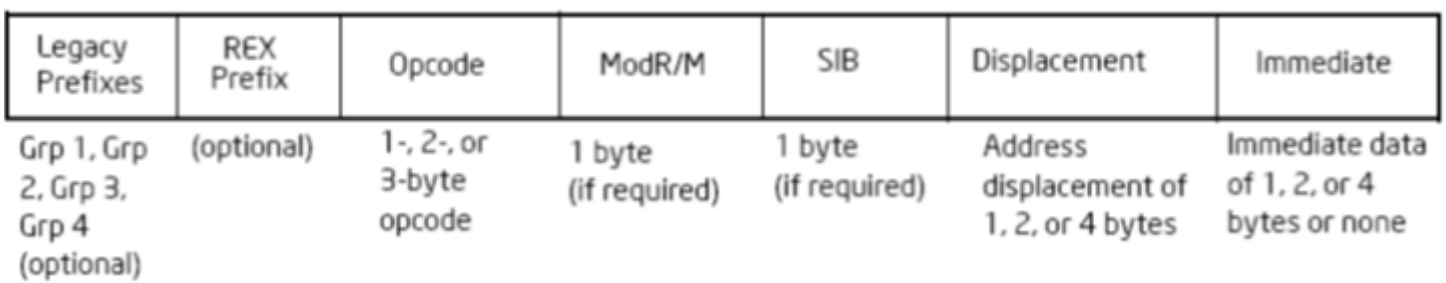


- кодирование AVX/AVX2 инструкций

EVEX префикс

- кодирование AVX512 инструкций

Поле смещения и явного значения



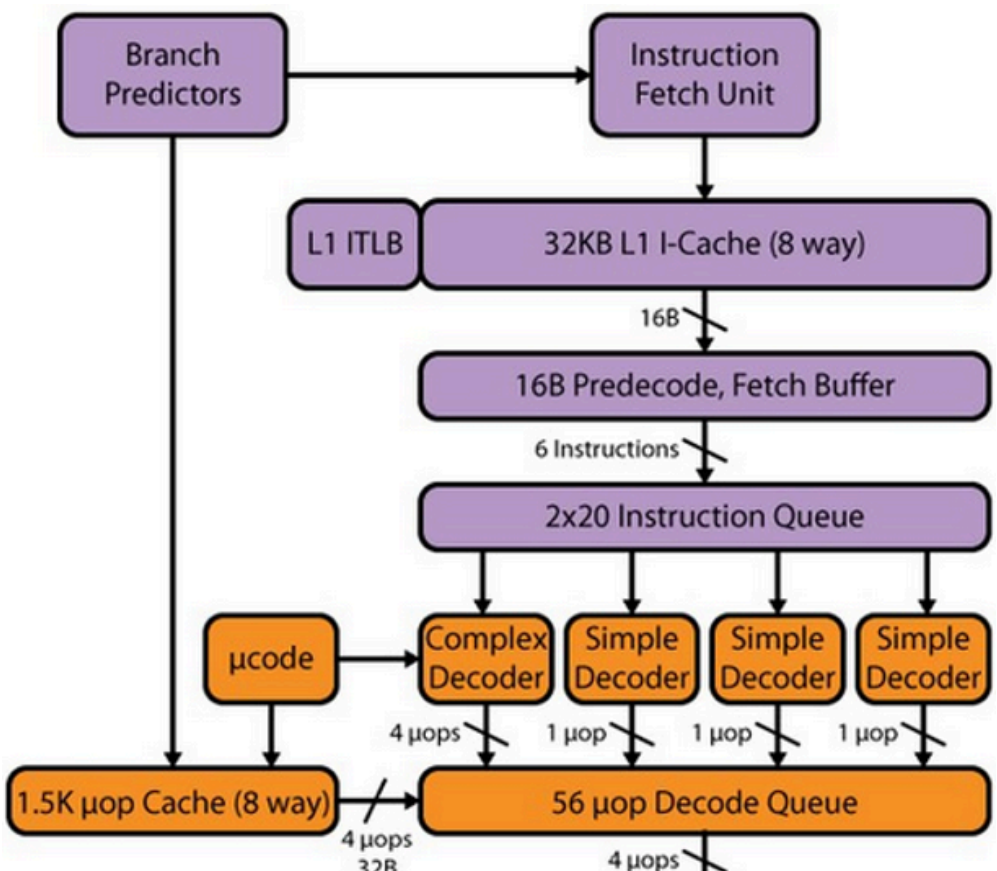
Legacy Prefixes	REX Prefix	Opcode	ModR/M	SIB	Displacement	Immediate
Grp 1, Grp 2, Grp 3, Grp 4 (optional)	(optional)	1-, 2-, or 3-byte opcode	1 byte (if required)	1 byte (if required)	Address displacement of 1, 2, or 4 bytes	Immediate data of 1, 2, or 4 bytes or none

- в 64-х битном режиме поле смещения и явного значения могут быть по 8 и 16 бит. В отличие от x86-32 в инструкции может присутствовать только один из них

Реальность для x86

- как всегда:
  - и сложнее, и проще одновременно
  - ещё более запутанно
- к какому типу архитектуры относится x86?
  - CISC — ISA для x86
  - RISC — ядро процессора (микрокод)

Реальность для x86: Intel Haswell



Микрокод

- программа, преобразующая машинный код в последовательность микроопераций, выполняемых микропроцессором
- примеры:
  - операции деления на уровне микроопераций часто не поддерживаются
  - пример:
    - операция делегирования на уровне микрокода операции деления

Микрооперации (uops)

- транслируется любая ассемблерная инструкция:
  - ADD EAX,EBX
    - 1 микрооперация
  - ADD EAX,[MEM1]
    - до 2 микроопераций
    - чтение из памяти во временный регистр
    - сложение 2 операндов
  - ADD [MEM1],EAX
    - до 3 микроопераций
    - чтение из памяти
    - сложение 2 операндов
    - запись в память

Интересные материалы

- Префиксы в системе команд x86-32
- Дизассемблер своими руками
- Микрокод в ассемблерных инструкциях процессоров x86
- Кодирование команд
- Stop the instruction set war (для разъяснения)